

实现 AMD 百亿亿次级异构处理器愿景

Realizing the AMD Exascale Heterogeneous Processor Vision

ISCA 2024 · 产业产品论文 · 中文全译

Alan Smith, Gabriel H. Loh, Michael J. Schulte, Mike Ignatowski, Samuel Naffziger, Mike Mantor, Mark Fowler, Nathan Kalyanasundharam, Vamsi Alla, Nicholas Malaya, Joseph L. Greathouse, Eric Chapman, Raja Swaminathan

Advanced Micro Devices, Inc. (AMD 公司)

译本说明：依据本地 isca2024_exascale.pdf 作者版本翻译，保留原文的技术范围、时态、数据和参考文献编号。正文、图题、表格和脚注译为中文；原始图像及代码保留，图内英文另附中文对照。参考文献保留完整原始著录，并附中文题名。译文重新分页，非 AMD 或 ACM 官方译本。原文中疑似引用编号不一致之处不作擅自更正。

原文著录：Alan Smith et al. Realizing the AMD Exascale Heterogeneous Processor Vision. ISCA 2024, Industry Product. 作者版本，14 页。

版权提示：© 2024，版权由所有者 / 作者持有，出版权许可给 ACM。允许在首页保留本声明及完整引文、且不以营利或商业利益为目的的个人或课堂复制；须尊重第三方内容版权。注明来源的摘要获准；其他复制、再版、服务器发布或列表再分发须预先许可和 / 或付费，联系 Permissions@acm.org。本作者版本仅供个人使用，不得再分发。完整声明见译本末尾。

缩写表（译本阅读辅助）

缩写	英文全称	中文含义
AI	Artificial Intelligence	人工智能
APU	Accelerated Processing Unit	加速处理单元
EHP	Exascale Heterogeneous Processor	百亿亿次级异构处理器
HPC	High-Performance Computing	高性能计算
CPU	Central Processing Unit	中央处理单元
GPU	Graphics Processing Unit	图形处理单元
HBM	High-Bandwidth Memory	高带宽内存
SoC	System-on-a-Chip	片上系统
DOE	Department of Energy	美国能源部
ML	Machine Learning	机器学习
LLM	Large Language Model	大语言模型
IOD	I/O Die	I/O 裸片
CCD	CPU Complex Die	CPU 复合体裸片
XCD	Accelerator Complex Die	加速器复合体裸片
GCD	Graphics Compute Die	图形计算裸片
CU	Compute Unit	计算单元
ACE	Asynchronous Compute Engine	异步计算引擎
IF	Infinity Fabric	AMD 互连架构
NoC	Network-on-Chip	片上网络
LDS	Local Data Share	本地数据共享存储
SRAM	Static Random-Access Memory	静态随机存取存储器
DRAM	Dynamic Random-Access Memory	动态随机存取存储器
DDR	Double Data Rate	双倍数据速率
TSV	Through-Silicon Via	硅通孔
HI	Heterogeneous Integration	异构集成
EFB	Elevated Fanout Bridge	高架扇出桥
USR	Ultra-Short Reach	超短距
PHY	Physical Layer	物理层 / 物理层接口
SerDes	Serializer/Deserializer	串行器 / 解串器
BPM	Bond Pad Metal	键合焊盘金属
BPV	Bond-Pad Via	键合焊盘通孔

缩写	英文全称	中文含义
RDL	Redistribution Layer	重布线层
P/G	Power/Ground	电源 / 地
TX / RX	Transmit / Receive	发送 / 接收
ISA	Instruction Set Architecture	指令集架构
AVX	Advanced Vector Extensions	高级向量扩展
HSA	Heterogeneous System Architecture	异构系统架构
AQL	Architected Queueing Language	架构化队列语言
API	Application Programming Interface	应用程序编程接口
BLAS	Basic Linear Algebra Subprograms	基础线性代数子程序
LAPACK	Linear Algebra Package	线性代数程序包
NUMA	Non-Uniform Memory Access	非统一内存访问
NPS	NUMA Nodes Per Socket	每插槽 NUMA 节点数
PCIe	Peripheral Component Interconnect Express	高速外设组件互连
SR-IOV	Single Root I/O Virtualization	单根 I/O 虚拟化
VF	Virtual Function	虚拟功能
TDP / TBP	Thermal Design Power / Total Board Power	热设计功耗 / 板卡总功耗
FLOPS	Floating-Point Operations Per Second	每秒浮点运算次数
FP / INT	Floating Point / Integer	浮点 / 整数, 后缀表示位数
BF16	Brain Floating Point 16	16 位脑浮点格式
TF32	TensorFloat-32	TensorFloat-32 数值格式
I/O	Input/Output	输入 / 输出
IP	Intellectual Property	知识产权; 此处指可复用设计模块
LGA	Land Grid Array	平面栅格阵列
NIC	Network Interface Card	网络接口卡
OCP	Open Compute Project	开放计算项目
NVMe	Non-Volatile Memory Express	非易失性内存高速接口
USB	Universal Serial Bus	通用串行总线
HPCG	High Performance Conjugate Gradients	高性能共轭梯度基准
CDNA	Compute DNA	AMD 计算架构名称

摘要

AMD 此前详细回顾了其百亿亿次级计算研究历程：从最初的目标和要求，到面向高性能计算（HPC）的加速处理单元（APU）愿景的形成与演进。这一 APU 被称为百亿亿次级异构处理器（Exascale Heterogeneous Processor, EHP）。在那项工作的最后，研究所得被融入 Frontier 超级计算机的节点架构。Frontier 是世界上第一台百亿亿次级计算机。不过，尽管 Frontier 节点架构体现了 EHP 概念的许多特征，当时的先进异构集成能力还不够成熟，尚不足以实现我们面向 HPC 和人工智能（AI）的完全集成式 APU 愿景。本文将继续讲述 EHP 的故事：深入分析为什么 APU 在我们第一代百亿亿次级架构诞生时并非合适的方案，早期 EHP 概念存在哪些不足，以及 AMD 如何进一步将这一概念发展为 AMD Instinct™ MI300A APU。MI300A 汇集了 AMD 多年来在先进封装技术、APU 硬件和软件方面的成果，也是我们卓有成效的芯粒策略的下一步。它既为百亿亿次级计算提供了突破性的设计，也满足了新兴大语言模型和生成式 AI 应用的需求。

I. 引言

十多年前，美国能源部（DOE）提出了一组雄心勃勃的目标，为其百亿亿次级计算计划提供指导 [7]。DOE 尤其担忧，摩尔定律 [23] 和登纳德缩放 [9] 走向终结、功耗、内存墙 [43]，以及大规模系统的可靠性等技术挑战，会显著推迟百亿亿次级超级计算机的问世。DOE 组织了一系列公私合作的先进研究项目，以支持并加快实现百亿亿次级计算能力所需的前期研究。AMD 入选并参与了这些研究计划。

我们在 ISCA 2023 产业分会论文 [22] 中，详细介绍了 AMD 百亿亿次级研究愿景的关键方面、这些想法如何演进，以及它们最终如何促成橡树岭国家实验室的 Frontier 成为世界上第一台百亿亿次级超级计算机 [28][29][30]。我们的研究愿景提出了“百亿亿次级异构处理器”（EHP）：一种高性能 APU，将 CPU 和 GPU 计算单元、多堆叠高带宽内存（HBM），以及其他片上系统（SoC）组件集成到同一个物理处理器封装内。由于本文将讨论的多种因素，最终部署在 Frontier 中的节点架构虽然在逻辑组织上体现了 EHP 的许多关键架构特征，却由彼此独立的分立封装组成。

劳伦斯利弗莫尔国家实验室宣布，其即将推出的 El Capitan 超级计算机将采用 AMD Instinct™ MI300A APU [37]。MI300A APU 将计算和内存组件结合在一个先进封装内，实现并体现了最初的 EHP 概念。本文从内部设计视角介绍：AMD 为什么没有在第一代百亿亿次级计算机中采用 EHP；原始 EHP 概念仍面临哪些挑战；AMD MI300A APU 的技术细节，以及它最终如何超越 EHP；AMD 芯粒策略如何演进为模块化 3D 芯粒平台，并由此支持 AMD Instinct™ MI300X 加速器，以应对机器学习（ML）大语言模型（LLM）和生成式 AI 工作负载迅速增长的计算需求。

II. 背景与上下文

本节简要回顾 AMD EHP 概念，为第 III 节讨论第一代 AMD 百亿亿次级方案为何未采用 EHP 提供必要背景。关于最初 EHP 和 Frontier 超级计算机的完整发展历程，请参阅我们此前的回顾论文 [22]。

A. EHP 概述

最初的 EHP 研究方案经历了多个版本的变化。变化依据包括不断更新的技术能力与限制信息、与产品路线图的契合程度，以及项目整体的性能和功耗目标。在 HPC 超级计算机场景中，APU 很有吸引力，因为 GPU 组件能够在每瓦功耗和每单位体积（m³）下提供很高的计算吞吐量，从而在数据中心计算密度和功耗限制内尽可能提高性能。CPU 组件仍然至关重要：既要处理不规则或难以在 GPU 上并行化的代码段（阿姆达尔定律仍须考虑），又要分派和协调 GPU 计算任务。将 CPU 和 GPU 放入同一封装，还能缩短数据传输延迟，提高组件间带宽，并降低同步与协调开销。

EHP 还提出采用封装内 HBM，以支持 CPU 与 GPU 之间的“零拷贝”内存分配与管理。CPU 可以直接在 HBM 中初始化数据，随后分派给 GPU 的内核也可以直接访问同一份物理内存，无需像 CPU 与 GPU 采用分立封装时通常那样，将数据从主机内存显式复制到设备内存。除了避免复制所带来的性能和功耗收益，这种统一共享内存架构也改善了可编程性，因为用户不再需要安排 CPU 与 GPU 之间的往返数据复制。把所有组件置于同一封装，还为根据工作负载特征在组件间动态调整功耗分配提供了新的机会。

图 1(a) 展示了 EHP 的“版本 3”（EHPv3），其采用有源硅中介层，与计算芯粒和 HBM 进行 3D 堆叠 [20][31][42]。该方案大幅采用堆叠的主要原因，是在处理器封装内尽可能容纳更多计算单元和内存。根据当时对第一代百亿亿次级架构推出时哪些技术能够成熟到支持大批量制造的判断，最终的 EHP “版本 4”（EHPv4）放弃了 3D 堆叠方式，转而采用有机基板上的 2D 芯粒 [27]，并结合 GPU 与 HBM 的 2.5D 无源硅中介层集成 [12]，如图 1(b) 所示。EHPv4 仍然保留了将 CPU、GPU 和 HBM 集成到同一封装内的关键特征，但与 EHPv3 相比，它对先进集成和封装技术的采用更为保守。

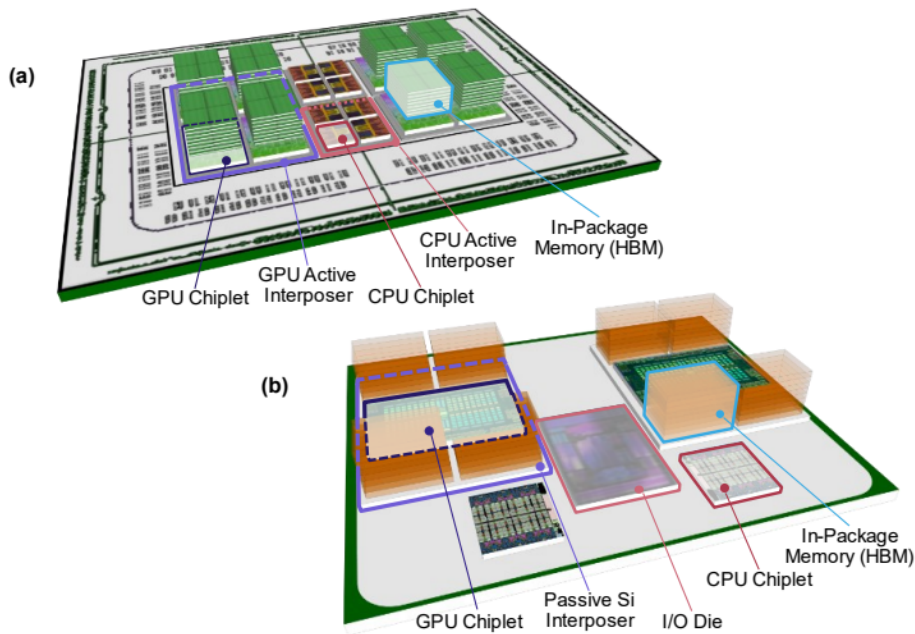


图 1. AMD 百亿亿次级计算前期研究中的 EHP 概念: (a) “版本 3” [42]; (b) “版本 4” [22]。

图内中文对照: GPU Chiplet: GPU 芯粒; CPU Chiplet: CPU 芯粒; GPU / CPU Active Interposer: GPU / CPU 有源中介层; In-Package Memory (HBM): 封装内内存 (HBM); Passive Si Interposer: 无源硅中介层; I/O Die: I/O 裸片。

B. Frontier 节点架构概述

由于第 III 节将讨论的多种因素，Frontier 超级计算机采用的节点架构没有使用 APU。图 2 展示了实际节点架构：一个 CPU 和四个分立 GPU 加速器，通过 AMD Infinity Fabric™ 一致性互连连接 [30][32]。正如我们此前的回顾论文 [22] 所述，Frontier 节点架构的底层组件实际上相当于四个 EHP 实例，通过一个公共 I/O 裸片 (IOD) 连接在一起；这些组件也已广泛部署于其他超级计算机和数据中心 [23]。

例如，图 2 中四种颜色的框各自包含两个 CPU 芯粒、两个大型 GPU 裸片和八个 HBM 堆叠，与一个 EHPv4 的计算和内存组件相对应。平坦的内存地址空间，加上 CPU 与 GPU 部分之间的缓存一致性，使这些组件呈现出类似 APU 的视图：架构上统一，物理实现上则位于不同封装中。

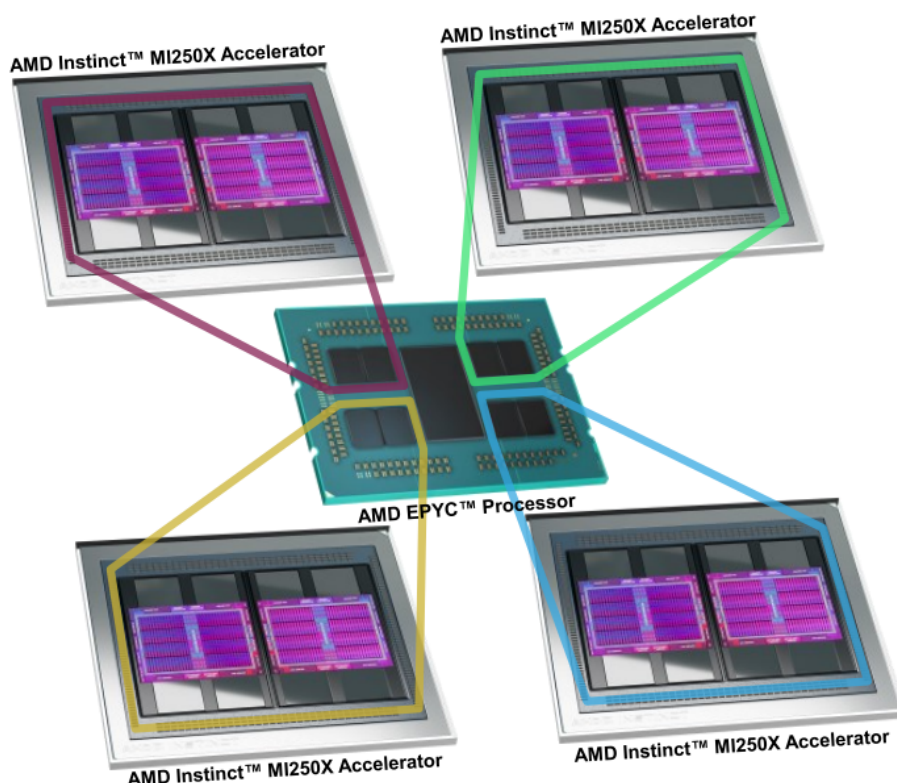


图 2. Frontier 节点架构中的计算组件。

图内中文对照：四个 AMD Instinct MI250X 加速器连接到一个 AMD EPYC 处理器。Accelerator：加速器；Processor：处理器。

C. 现代机器学习时代的到来

百亿亿次级计算计划启动时，重点是 HPC 和科学计算。在通往第一代百亿亿次级架构的这十年研究期间，业界见证了现代机器学习革命的爆发。这为我们的产品带来了新要求，AMD 也相应调整了愿景，力求为 HPC 和 ML 同时提供世界一流的性能与能效。

HPC 和 ML 有许多共同需求，尤其是极高的内存带宽和计算吞吐量。不过，ML 也提出了新要求，例如传统 HPC 不太强调的低精度运算，以及容纳不断增大的模型所需的更大内存容量。当然，目前也已有大量工作致力于将 ML 技术应用到 HPC 场景 [34]。市场对世界一流 ML 产品的需求，促使 AMD 在第二代百亿亿次级计划中同时支持这两类市场需求。这要求 AMD 将芯粒策略推进到新的 3D 模块化水平。

III. EHP：接近目标，但仍差一步

我们首先回顾 AMD 为什么没有为 Frontier 超级计算机构建 EHPv3 或 EHPv4。面向 HPC 的 APU 整体愿景在当时和现在都具有技术合理性，但当时业界的异构集成能力尚未成熟到足以将我们的百亿亿次级 APU 概念大批量产品化。本节也为理解 MI300A 处理器的设计方法作铺垫。

A. EHPv3 对技术的激进要求

在某些方面，EHPv3 比 EHPv4 更理想，因为其集成密度明显更高。尽管 AMD 已成功在采用 V-Cache™ 技术的 CPU 产品中使用混合键合 3D 堆叠 [14]，EHPv3 的堆叠方式仍远比 V-Cache 产品激进。

作为对照，AMD EPYC™ 和 Ryzen™ 处理器采用的 V-Cache 技术，是将一个 SRAM 芯粒堆叠在 CPU 芯粒上，其尺寸仅为数十平方毫米 [26][44]，如图 3(a) 所示。EHPv3 的 3D 堆叠装配流程则要复杂得多。首先，图 3(b)

中每个 GPU 芯粒的占地面积，都要不小于一个 HBM 堆叠的占地面积（每个堆叠约 100 mm²）[40]。随后，四个这样的 GPU 芯粒需要堆叠在一个有源中介层裸片上，因此该中介层必须大于 400 mm²，才能提供足够空间。除此之外，每个 GPU 芯粒上还需要再堆叠 HBM。

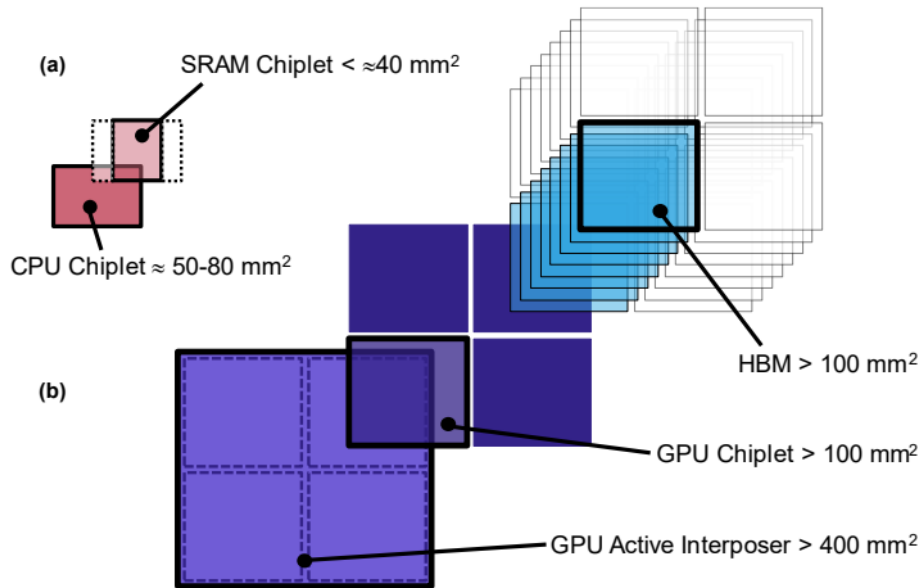


图 3. (a) AMD V-Cache 技术中堆叠于 CPU 芯粒上的 SRAM 芯粒；(b) EHPv3 GPU 复合体的堆叠组件。示意图大致按比例绘制。

图内中文对照：SRAM Chiplet：SRAM 芯粒，图示标注为小于约 40 mm²；CPU Chiplet：CPU 芯粒，约 50-80 mm²；HBM：大于 100 mm²；GPU Chiplet：GPU 芯粒，大于 100 mm²；GPU Active Interposer：GPU 有源中介层，大于 400 mm²。

使 EHPv3 整体装配流程成熟所需的时间，无法与 Frontier 的进度相匹配。主要原因在于：需要增加的工艺步骤、必须单独处理和测试的裸片或堆叠数量、为实现两层以上堆叠而增加的裸片减薄和硅通孔（TSV）制造步骤，以及整个结构更大的尺寸。通过这种 3D 结构散热的难度，也会超过当时的冷却能力。Frontier 最初计划确定后，AMD 与晶圆代工合作伙伴继续演进并完善 3D 堆叠技术，推出了多款 3D 堆叠 Ryzen 和 EPYC 产品。后续超越 Frontier 的方案利用了这些增强后的能力。

除了 EHPv3 内部组件的 3D 堆叠，多个由有源中介层、芯粒和 HBM 组成的复合体，还必须共同封装。将多种集成和封装技术结合起来的异构集成（HI）近年来受到广泛关注 [19]，而 AMD 多年来已经在混合使用多种裸片堆叠和先进封装形式。EPYC 和 Ryzen V-Cache 处理器把 3D 混合键合组件与传统有机基板 2D 芯粒封装结合起来。AMD Instinct™ MI250X 加速器则结合了两个大型 GPU 芯粒、2.5D 高架扇出桥硅裸片，以及基于微凸点的 3D HBM 堆叠 [1][2][36]。然而，这些方案所要求的异构集成规模和尺寸，都无法与 EHPv3 相比；在 Frontier 的时间窗口内，EHPv3 的复杂度超出了实际可承受的范围。

B. EHPv4 的不足

EHPv4 为更好地赶上 Frontier 的时间安排，放弃了 EHPv3 中许多较激进的封装概念，但仍保留了多项有吸引力的特征，能够构成性能强、效率高的 HPC APU。不过，最终的 EHPv4 仍有若干设计问题和未解决的技术挑战，使其无法完全满足需要。

EHPv4 尝试复用主流 EPYC 服务器处理器的 IOD。通常而言，复用硅设计是可取的 [27]，但前提是应用场景相匹配。对 EHPv4 而言，来自服务器的 IOD 带来了一些问题。首先，服务器 IOD 上裸片间接口的尺寸和位置，迫使 EHPv4 采用不够理想的整体芯粒拓扑。两个 GPU 部分最终相隔较远，当一个 GPU 裸片访问另一侧的 HBM 堆叠时，这段距离会限制带宽并增加功耗，即图 4 中的 ①。如此长的距离无法有效利用无源硅中介层或高架扇出桥（EFB）互连 [36] 等更高带宽的封装方案；这类方案通常只用于直接相邻或紧靠的裸片，例如 GPU 与 HBM。因此，设计只能采用类似 MI250X 中 GCD 之间所用的 2D AMD Infinity Fabric™（IF）链路。

服务器 IOD 的芯粒间 IF 链路也不适合我们的 HPC APU。服务器 IOD 的 IF 链路 ② 原本按照 DDR 内存的带宽水平配置，因此在以封装内 HBM 为中心的系统可能成为瓶颈。

整体芯粒拓扑还使 CPU 芯粒到 HBM 的路径较长 ③：请求必须经过两跳裸片间 IF 链路，再加上 IOD 和 GPU 内部的数据互连网络 / 片上网络传输，才能到达内存；返回数据还要再次付出同样的代价。

IOD 的利用率也不高，因为第四代 EPYC IOD 的十二条 IF 链路 [41] 中，有几条 ④ 无法连接任何芯粒，因而浪费了裸片面积。同样，GPU 与 HBM 复合体的位置会阻挡 IOD 某些 DDR 内存通道和 I/O 通道的信号引出路径 ⑤，使这些接口占用的裸片面积也无法利用。在追求尽可能高性能的方案中，应尽量充分利用宝贵的封装面积，但 EHPv4 的封装内仍有多处空白区域 ⑥。复用服务器 IOD 还意味着，它与其他芯粒的连接只能采用基于 2D 有机基板的封装，因为未经修改的服务器 IOD 不具备例如 EFB 链路所需的接口。这使 EHP 数据传输的带宽和能耗都不够理想。

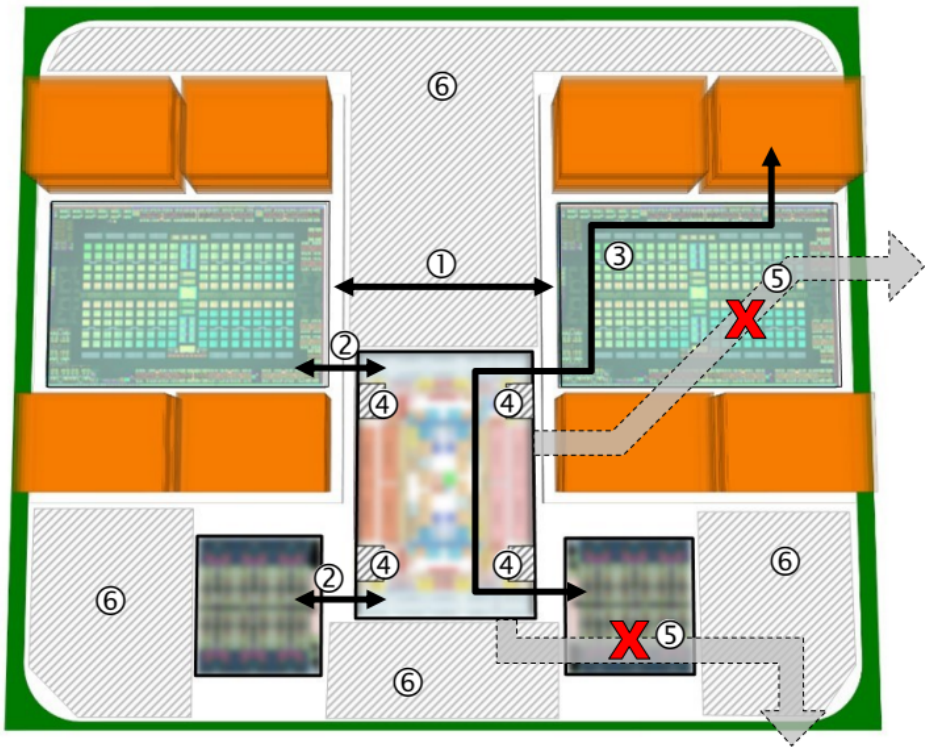


图 4. EHPv4 仍存在的若干挑战。

图内中文对照：① 两侧 GPU 间距过大；② IF 链路按 DDR 带宽配置；③ CPU 到 HBM 路径较长；④ 部分 IF 链路闲置；⑤ DDR / I/O 信号引出路径受阻；⑥ 封装内未利用的区域。编号含义对应正文。

因此，将 EPYC CPU 与 Instinct MI250X GPU 组合为一致性异构节点架构，是我们第一代百亿亿次级架构的正确选择 [22][32]。不过，Frontier 方案成功结合了多种先进封装技术，而这些产品带来的经验又增强了信心、降低了风险，为面向 HPC 和 ML 的 APU 铺平了道路 [23][24]。

IV. MI300A 的架构组织

AMD Instinct™ MI300A APU 在 EHP 概念基础上发展而来。本节介绍 MI300A 的架构组织，第 V 节讨论其物理构造。

A. 概述

MI300A APU 由 CPU 和 GPU 芯粒、共享的 AMD Infinity Cache 内存侧缓存、八个 HBM 堆叠、I/O 接口，以及负责这些组件间数据传输的 Infinity Fabric (IF) 互连组成。图 5 展示了 MI300A APU 的框图。

MI300A 共有六个加速器复合体裸片 (XCD)，提供 APU 中高度并行且能效较高的 GPU 计算引擎。“GPU”这一名称并不十分准确，因为 XCD 所采用的 CDNA™ 3 架构已经优化掉了用于像素处理和光线追踪的传统图形专用硬件。MI300A APU 还采用三个 CPU 复合体裸片 (CCD)，提供 CPU 核心。

XCD 和 CCD 全部连接到 IF 互连，后者承担 SoC 的片上网络 (NoC) 功能。由于 MI300A 的物理构造 (第 V 节)，这个 “NoC” 实际上跨越多个芯片。IF 将内存请求路由到系统的 128 个 HBM 内存通道及 I/O 接口，并承载返回数据。每个内存通道还配有一片 2 MB 的 Infinity Cache，总容量为 256 MB。以下小节将进一步介绍各个组件。

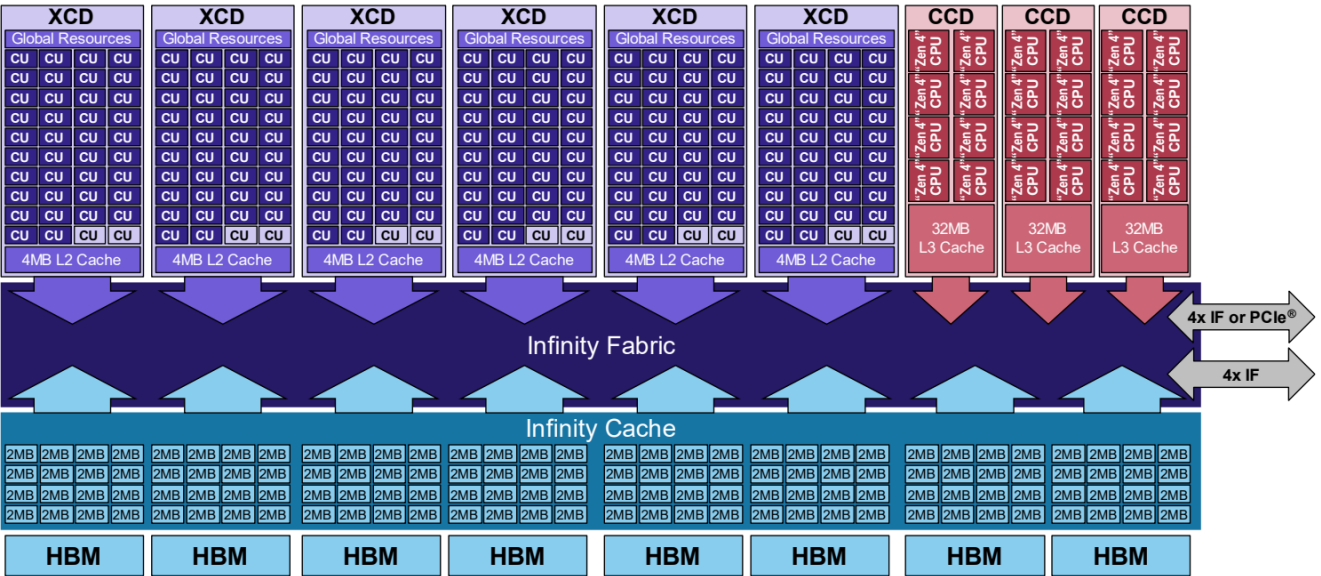


图 5. MI300A 加速处理单元的逻辑 / 架构框图。

图内中文对照：Global Resources：全局资源；CU：计算单元；4MB L2 Cache：4 MB L2 缓存；32MB L3 Cache：32 MB L3 缓存；Zen 4 Core：“Zen 4” 核心；Infinity Fabric：IF 互连；Infinity Cache：内存侧缓存；4x IF or PCIe：四条 IF 或 PCIe 链路；4x IF：四条 IF 链路。每个 HBM 通道对应 2 MB 缓存切片。

B. XCD

MI300A 的 XCD 采用 5 nm 工艺，每个 XCD 提供 38 个计算单元 (CU)，整个 APU 合计 228 个 CU。需要说明的是，每个 XCD 在物理上实现了 40 个 CU，但为了提高良率，只启用其中 38 个；也就是说，最多允许两个 CU 有缺陷，如图 5 中浅色 CU 所示。

每个 XCD 包含共享的全局资源，包括调度器、硬件队列，以及四个异步计算引擎 (ACE)。ACE 将计算着色器工作组发送给 CU。一个 XCD 内的 CU 共享 4 MB L2 缓存，用于汇集该裸片上的全部内存流量。

每个 CU 都是高度多线程化的处理器，包含取指单元、调度器、标量 / 向量 / 矩阵执行单元，以及加载 / 存储流水线。其内存层次结构起始于 32 KB L1 数据缓存和 64 KB 本地数据共享存储 (LDS)。L1 数据缓存的缓存行大小增至 128 B，数据缓存的输入和输出总线宽度也相应增加，使缓存带宽相较 CDNA 2 架构 [2] 实际翻倍。每两个 CU 共享一个 64 KB、8 路组相联指令缓存。GPU 工作负载中，最常见的情况是多组 CU 执行同一指令流，因此共享指令缓存可以提高命中率，同时对裸片面积的影响很小。

与 MI250X 加速器采用的 CDNA 2 架构相比，MI300A 的 CDNA 3 架构显著提高了矩阵核心处理训练和推理所用机器学习数据类型的性能。表 1 列出了 CDNA 2 和 CDNA 3 架构每个 CU 每时钟周期的运算次数，也突出显示了 MI300A XCD 新增的 FP8 数据类型支持。

表 1. MI250X 中 CDNA 2 CU 与 MI300A 中 CDNA 3 CU 的峰值运算次数：每 CU、每时钟周期。

架构	向量 FP64	向量 FP32	矩阵 FP64	矩阵 FP32	矩阵 TF32	矩阵 FP16	矩阵 BF16	矩阵 FP8	矩阵 INT8
CDNA 2	128	128	256	256	不适用	1024	1024	不适用	1024
CDNA 3	128	256	256	256	1024	2048	2048	4096	4096

表中未体现的是，CDNA 3 矩阵核心还支持 4:2 稀疏性；在这种条件下，对于 FP8 和 INT8，峰值吞吐量最高可达每 CU 每周期 8192 次运算。CDNA 3 的更多细节见 [5]。

C. CCD

MI300A APU 复用 AMD 第四代 EPYC CPU [4] 中的 “Zen 4” CCD [26]，并为混合键合作了修改。与 XCD 一样，CCD 采用 5 nm 工艺。每个 CCD 提供八个 “Zen 4” 核心，共享 32 MB L3 缓存。MI300A APU 有三个 CCD，总计 24 个 “Zen 4” 核心。

“Zen 4” 微架构的完整细节可参阅已有工作 [14][41]。与上一代 “Zen 3” 相比，“Zen 4” 的几个主要改进包括：每核心 L2 缓存翻倍至 1 MB；提高分支预测准确率；扩大多个微架构结构，包括操作缓存、退休队列、整数和浮点寄存器文件；提高时钟频率和每瓦性能；在指令集架构 (ISA) 中加入对 AVX 512 指令的支持。CCD 执行全部传统 x86 代码，包括操作系统所需的所有代码，以及用户代码中没有卸载到 XCD 的部分。

D. 封装内 Infinity Fabric 与内存组织

CCD 和 XCD 共享基于 HBM 的统一内存系统。两类处理器都可以直接通过加载和存储访问 HBM，无需像采用分立 GPU 的系统那样复制数据；在后者中，GPU 拥有独立于主机 CPU 内存的显存。CPU 可以初始化内存，然后直接在 GPU 组件上启动内核。这既有性能收益，也改善了可编程性，第 VI 节会进一步讨论。

CPU 使用与 EPYC CPU 同类、基于探测过滤器的一致性协议，与所有 CPU 和 GPU 保持硬件一致性。GPU 与其他插槽中的 GPU 之间采用软件一致性，以减少硬件一致性带宽需求；同一插槽内部的 GPU 则采用基于目录的硬件一致性协议，其协议比 CPU 使用的协议稍简单。

如图 5 所示，公共 Infinity Fabric 互连负责 CCD、XCD 与八个 HBM 堆叠之间的数据传输。MI300A APU 合计提供 128 GB 内存容量、128 个内存通道，理论峰值带宽约为 5.3 TB/s。IF 与 HBM 通道采用相对细粒度的交错方式：每段连续 4 KB 物理地址映射到同一个 HBM 堆叠，然后根据物理地址散列方案选择下一个 HBM 堆叠。IF 也连接 I/O 接口，第 VIII 节将作详细讨论。

128 个内存通道中的每一个，都配有一片 2 MB 的 Infinity Cache，总容量为 256 MB。Infinity Cache 作为内存侧缓存运行，因此无需参与缓存一致性事务。其主要作用是放大 HBM 的有效带宽，最多可提供 17 TB/s 带宽。Infinity Cache 还具有硬件预取器，以帮助降低内存访问延迟。

V. MI300A 的物理构造

MI300A APU 由约 1460 亿个晶体管组成，不计 HBM，在多个方面推动了制造能力的边界。本节讨论 MI300A APU 所采用的 AMD 芯粒、3D 堆叠、封装、IP 复用和模块化架构方法上的进展。

A. MI300A 的异构集成技术

MI300A 架构所包含的逻辑数量，需要相当于多个光刻掩模曝光区域的晶体管规模才能实现。我们此前的工作介绍了如何利用芯粒设计，把逻辑上庞大的架构分解到多个物理上独立的硅芯粒中 [27]。然而，如前文讨论 EHPv4 时所述，仅靠 2D / 2.5D 布置的芯粒，仍不足以提供所需的集成密度，无法把 MI300A 的所有组件放入尺寸合理的封装。

为此，MI300A 采用了多项技术的激进组合，如图 6 所示。各个 XCD 和 CCD 芯粒通过 3D 混合键合，与多个有源中介层 IOD 进行垂直连接。混合键合采用直接接触键合，使两个裸片上的金属键合焊盘在原子层面融合，而不是逐个放置微凸点。这形成了高密度的垂直互连：AMD V-Cache 产品和 MI300A 的间距均为 $9\ \mu\text{m}$ ；相较于基于微凸点的 3D 堆叠，它还具有更好的导热性能 [44]。剖面图展示了 MI300A 内全部无源和有源组件的堆叠结构，包括保持整个复合体机械稳定性所需的结构硅和载体硅组件。为便于说明，剖面图中的 HBM 被移到了“两侧”。

与八个 HBM 堆叠连接并提供全部 I/O 接口，所需的芯片边缘接口长度会使单个 IOD 极其庞大，远超标准光刻曝光区域的尺寸。过去，AMD 将芯粒用于计算部分，例如 EPYC 和 Ryzen CPU 中的 CCD；MI300A 则采用类似的划分方式，使用四个独立 IOD。图 6 右侧的平面图可见这些 IOD，其中左下方 IOD 被突出显示；图 7 也展示了它们。

每个 IOD 顶部混合键合多个 CCD 或 XCD，然后与八个 HBM 堆叠一起通过硅中介层进行 2.5D 集成。相邻 IOD 之间的距离很短，因此我们能够采用功耗极低、延迟很低的超短距（USR）物理层接口（PHY）进行 IOD 间数据传输。相较传统串行器 / 解串器（SerDes）[27]，USR PHY 的单位面积带宽密度（Tbps/mm²）提高十倍以上，同时降低功耗，达到 0.4 mW/Gbps。USR 接口的最小微凸点间距为 $35\ \mu\text{m}$ 。USR 接口合计提供数 TB/s 的带宽（图 7），使 HBM 的访问效果如同 Infinity Fabric 实现在一个单片 IOD 上。

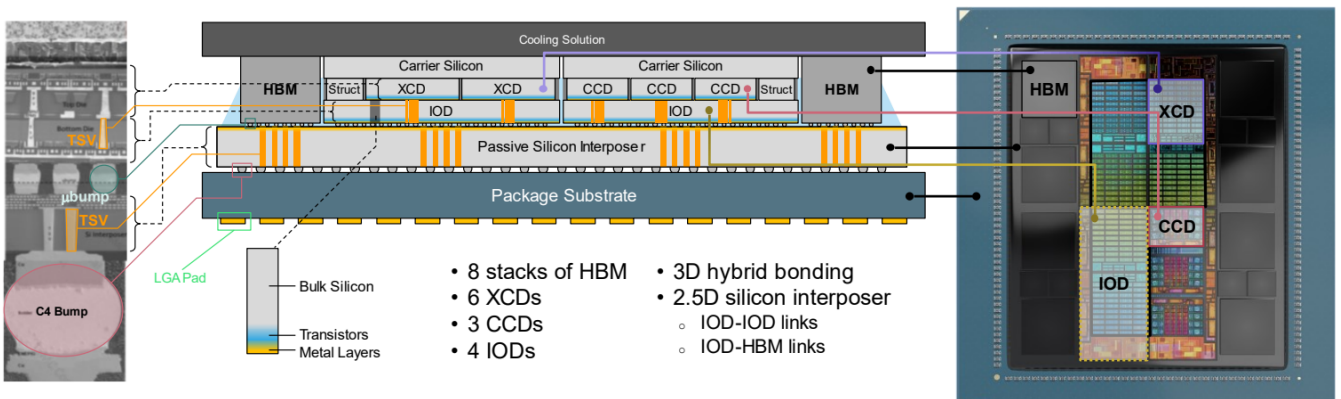


图 6. MI300A APU 的异构集成构造细节，结合了多项先进集成与封装技术、芯粒和 3D 内存堆叠。剖面图未按比例绘制。

图内中文对照：Cooling Solution：冷却装置；Carrier Silicon：载体硅；Struc / Structural Silicon：结构硅；Passive Silicon Interposer：无源硅中介层；Package Substrate：封装基板；Bulk Silicon：体硅；Transistors：晶体管；Metal Layers：金属层；Cu Bump：铜凸点； μbump ：微凸点；TSV：硅通孔；LGA Pad：平面栅格阵列焊盘。组成：8 个 HBM 堆叠、6 个 XCD、3 个 CCD、4 个 IOD；采用 3D 混合键合和 2.5D 硅中介层，后者承载 IOD-IOD 与 IOD-HBM 链路。

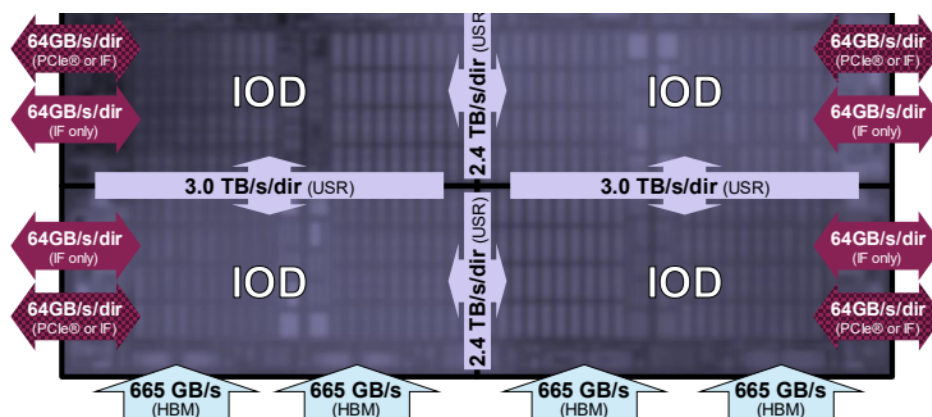


图 7. MI300A IOD 各类接口的带宽。

图内中文对照：/dir 表示每个方向。横向 IOD 间 USR 链路为每方向 3.0 TB/s，纵向为每方向 2.4 TB/s；图中各 HBM 接口为 665 GB/s；外部链路每方向 64 GB/s，分别为 PCIe 或 IF、仅 IF。

B. 3D 接口

XCD 专为 MI300A 设计，因此我们可以直接布置其 3D 接口，使其与下方 IOD 上的对应连接点对齐。

为复用硅设计，MI300A 对 AMD EPYC™ CPU 中使用的同款 CCD 进行了适配。在 EPYC 处理器配置中，CCD 使用面向 2D 有机基板封装的 IF SerDes 接口，如图 8(a) 中突出显示的部分。然而，MI300A 采用 3D 堆叠组织，因此需要增加用于混合键合堆叠工艺的键合焊盘金属（BPM）连接位置。图 8(a) 展示了我们如何在 CCD 布局的空白区域挤入 3D 接口。这也说明，基于 TSV 的 3D 接口比有机基板 2D 互连方案密集得多。在 CCD 内部，我们根据其所用于的产品，在 2D 和 3D 接口之间进行简单的接口复用选择。

IOD 一侧的对应 3D 接口被布置为与三个 CCD 对齐，如图 8(b) 所示。其中两个 CCD 相对另一个旋转了 180°，IOD 上的接口也作了相应旋转，以保持正确对齐。

无论一个 IOD 上堆叠的是 CCD 还是 XCD，MI300A 都采用同一套 IOD 物理设计。CCD 与 XCD 的尺寸不同、带宽和接口宽度不同，芯粒总数也不同：一个 IOD 上放置三个 CCD，或两个 XCD。为支持这些情况，IOD 实现了包含两类接口的超集，如图 8(c) 所示。当两个 XCD 堆叠在 IOD 上时，使用中间的深蓝色 TSV 接口组，而三组 CCD 接口被禁用，以虚线 / 阴影框表示。与 CCD 类似，图中一个 XCD 旋转了 180°，对应 TSV 接口也相应旋转。对 XCD、CCD 和 IOD 接口的精细协同规划，使 IOD 顶部的计算芯粒能够按模块替换。

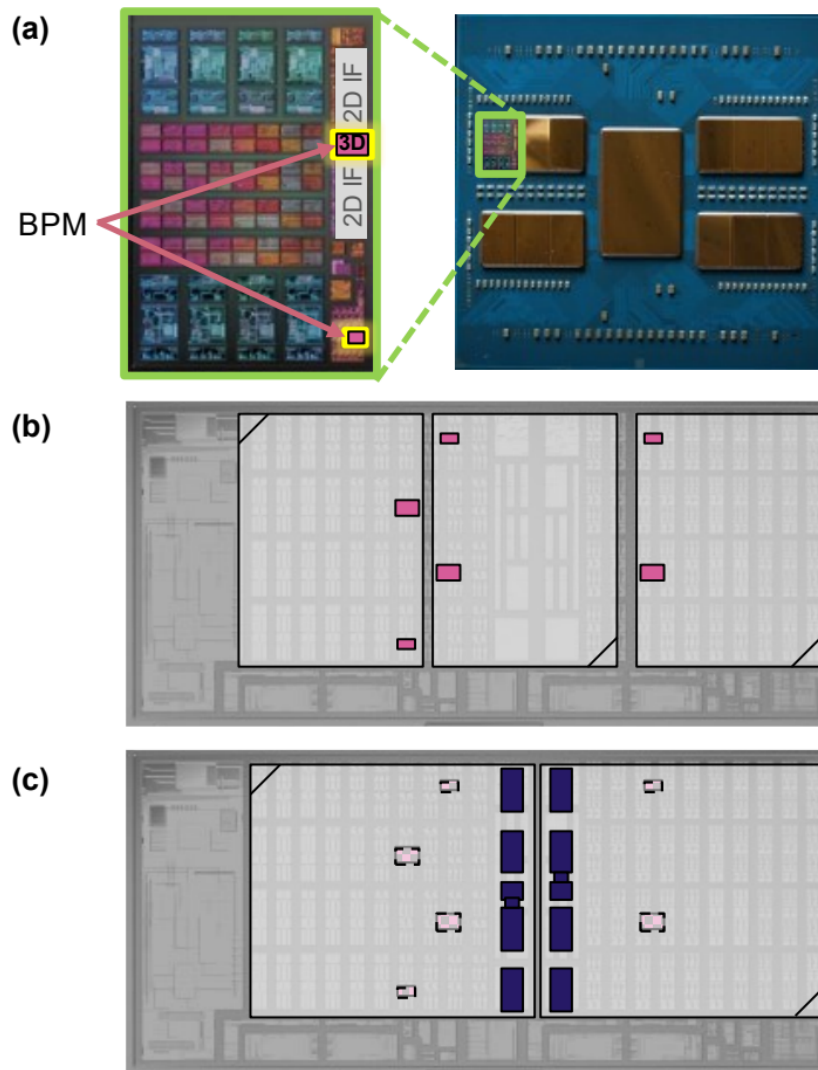


图 8. (a) CCD 上的 2D 和 3D 接口示意; (b) IOD 上的 CCD 3D 接口; (c) IOD 上的 XCD 3D 接口。

图内中文对照：2D IF：二维 IF 接口；3D：三维接口；BPM：键合焊盘金属。虚线框表示未启用的 CCD 接口。

C. IOD 镜像

MI300A 总计使用四个 IOD 硅实例。其中两个 IOD 是同一物理设计的“镜像”副本，仅作少量修改。这使两次非常相近的流片能够复用几乎全部布局规划、物理设计和工程工作。图 9 展示了四个 IOD：两个普通版本和两个镜像版本，每种 IOD 中各有一个旋转 180°。IOD 角落的“Instinct”标志和对齐标记用于辨别方向与镜像关系。

为使 IOD 彼此正确连接，镜像 IOD 中的 USR 发送 (TX) 和接收 (RX) 模块需要交换位置，如镜像 IOD B 和 IOD D 上的双向箭头所示。这样，一个 IOD 的每个 TX 模块都能与相邻 IOD 上对应的 RX 模块直接配对。这说明 IOD 彼此并非完全的几何镜像，但这类修改可以通过相对简单的设计算法调整来支持。

镜像 IOD 也给 3D 堆叠芯粒带来了一个有趣的接口挑战。IOD 的布局作了对称镜像，而 CCD 和 XCD 没有镜像版本。因此，接口设计必须保证，无论下方 IOD 是原始实例还是镜像实例，未经镜像的 CCD 和 XCD 都能正确落位并保持对齐。

图 9 左侧展示了分别堆叠在 IOD A 和 IOD B 上的两个 XCD；二者方向相同，均未经镜像。IOD A 上的红圈表示非镜像 IOD 实例中不使用的冗余 TSV 接口。当 IOD 被镜像，例如 IOD B 时，这些原本冗余的 TSV 就与未经镜像的 XCD 对齐，从而让同一 XCD 既能连接镜像 IOD，也能连接非镜像 IOD。

这种 TSV 冗余仅用于 3D 信号接口；电源 / 地 TSV 将在下一小节讨论。对 IOD、XCD 和 CCD 接口的精确对齐安排，使 MI300A 能够支持所需的各种 IOD 旋转或镜像组合，以及 CCD、XCD 的旋转组合。

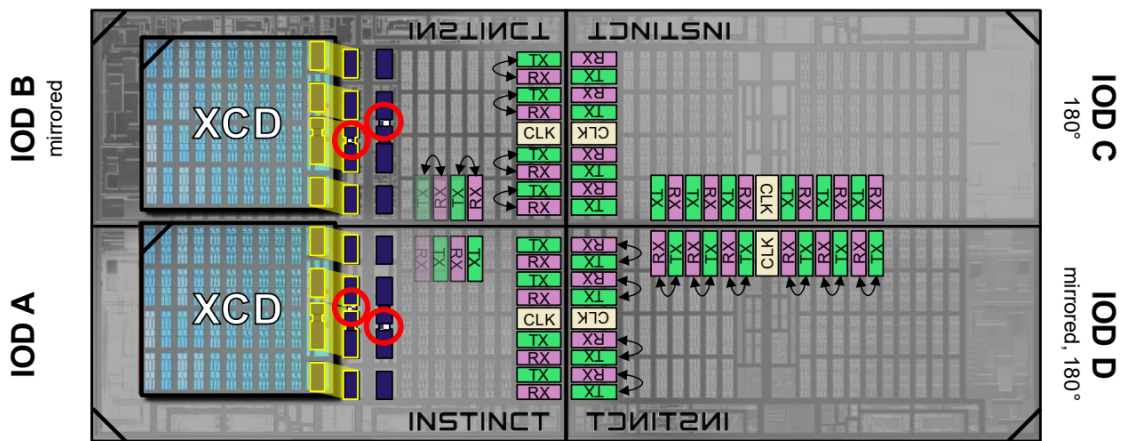


图 9. 复制 TSV（红圈）以对齐未经镜像的芯粒，以及通过 USR PHY 对齐实现 IOD 镜像。

图内中文对照：mirrored：镜像；rotated 180°：旋转 180°；TX：发送；RX：接收；CLK：时钟。IOD A 为普通方向，IOD B 为镜像，IOD C 为旋转 180°，IOD D 为镜像并旋转 180°。

D. 供电

前文重点讨论了在不同镜像与旋转组合下，如何适配传输数据和控制信号的 3D 接口。但在规划 IOD 中为芯粒供电的 TSV 时，也存在类似挑战。我们的方案采用均匀的电 源 / 地 (P/G) TSV 网格，并让 CCD 与 XCD 使用一致的网格。这在概念上很直接，但实现时需要大量前期规划，确保每一个 P/G TSV 在所有镜像和旋转的 IOD、CCD、XCD 组合中都能够对齐。最终的 IOD TSV 网格可以提供超过 1.5 A/mm² 的电流，同时保持很低的 I²R 损耗。

IOD 底部连接无源硅中介层的微凸点接口，除了向堆叠芯粒提供 1.5 A/mm² 电流外，还能额外提供 0.5 A/mm²，为 IOD 本身供电。当工作负载在计算主导阶段与内存密集阶段之间切换时，可以在 IOD 与计算芯粒之间沿垂直方向转移和重新分配功耗预算，下一小节将进一步讨论。

除了电源 / 地 TSV 的规划，Infinity Cache 阵列宏单元也需要与整体供电方案协同优化。阵列经过定制，其间距与 P/G TSV 条带之间的通道相匹配，从而能够放入其中，如图 10 所示。

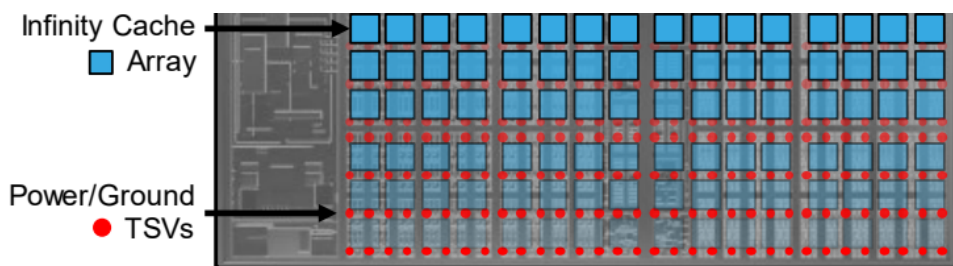


图 10. 芯粒供电 TSV 与 Infinity Cache SRAM 阵列宏单元的协同布局。

图内中文对照：Infinity Cache Array：Infinity Cache 阵列；Power/Ground TSVs：电源 / 地硅通孔。蓝色块表示阵列，红点表示电源 / 地 TSV。

虽然 MI300A 采用与 V-Cache 技术相同的基本混合键合工艺，但仍需要进一步优化堆叠流程。图 11(a) 展示了采用 V-Cache 技术的“Zen 3”代 CCD 的 3D 混合键合接口剖面 [44]。混合键合界面的 BPM 通过键合焊盘通孔 (BPV) 连接至 SRAM 裸片的顶层金属。然而在 MI300A 中，CCD 和 XCD 堆叠在顶部，如图 11(b) 所示，功率需求高于 V-Cache SRAM 裸片。MI300A 的 BPV 直接连接到铝重布线层 (RDL)，后者电阻更低，能够更

有效地向计算芯粒供电。

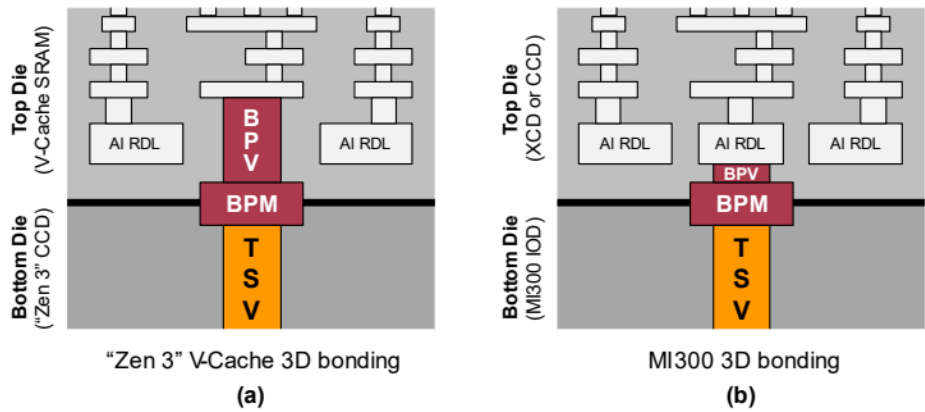


图 11. (a) 早期 V-Cache 实现中的 3D 键合接口; (b) MI300A APU 中的 3D 键合接口。

图内中文对照：Top Die：顶部裸片；Bottom Die：底部裸片；AI RDL：铝重布线层；BPV：键合焊盘通孔；BPM：键合焊盘金属；TSV：硅通孔。左图顶部是 V-Cache SRAM、底部是 Zen 3 CCD；右图顶部是 XCD 或 CCD、底部是 MI300 IOD。

E. 功耗管理与散热

采用 3D 堆叠时，功耗和热管理是系统整体物理设计中非常重要的因素。如上一小节所述，MI300A 加速器可以在不同物理组件间动态重新分配功耗预算。图 12(a) 给出了两种代表性的归一化功耗分布，分别对应计算密集型（GPU）和内存密集型工作负载。计算密集场景中，大部分功耗预算可以分给计算芯粒；内存密集场景中，更多功耗预算可以转移给内存系统、数据互连网络 and USR 链路。这形成了供电系统必须应对的两种极端工作条件。

除了供电系统，散热方案也必须适应这些差异很大的工作条件。图 12(b) 展示了 GPU 密集场景的热仿真结果，热点集中在 XCD 上。图 12(c) 则展示了内存密集型工作负载的情况。这里的温度分布更加混杂：热图中仍能辨认 XCD 的活动，但数据传输带来的功耗影响更加明显。模块外围可见八个 HBM PHY，而 USR PHY 也十分突出，因为它们在四个 IOD 之间传输数 TB/s 的数据。通过精细工程设计，协同规划 TSV 位置、功率密度及功耗分布，MI300A 实现了有效的功耗与热管理。

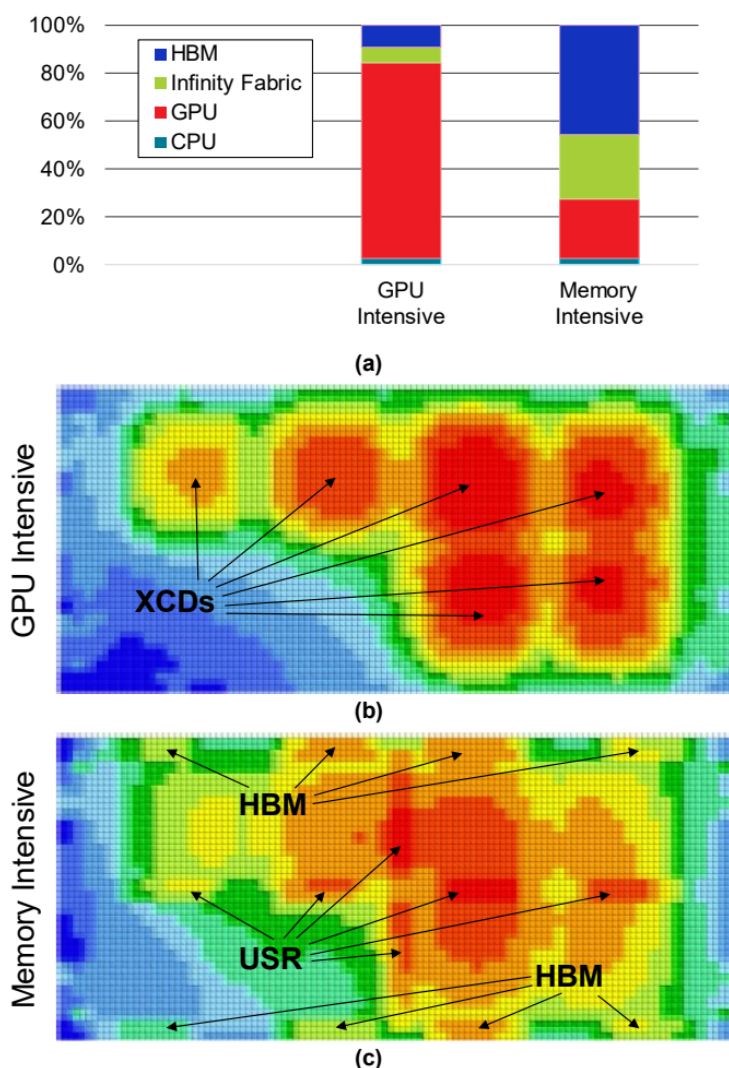


图 12. (a) 不同工作负载场景下的代表性功耗分布；(b) GPU 密集型与 (c) 内存密集型工作负载的热仿真预测。

图内中文对照：GPU Intensive：GPU 密集型；Memory Intensive：内存密集型。上图纵轴为功耗占比，图例依次为 HBM、Infinity Fabric、GPU、CPU。热图箭头分别标出 XCD、HBM 和 USR 所在区域。

F. 与 EHP 的比较

MI300A APU 最终设计的物理形态与早期 EHP 概念有很大不同，但也体现了最初为支持百亿亿次级计算而设想的许多特征。与 EHP 一样，MI300A 是单封装 APU，将异构 CPU、GPU 计算组件与统一的封装内内存系统结合起来。具体芯粒数量略有不同，但两者最终都采用每一个 CCD 对应两个 GPU 计算芯粒的比例，即 EHPv4 的 4:2 和 MI300A 的 6:3。EHP 与 MI300A 还采用了相同数量的 HBM 堆叠。EHPv3 也预见了最终采用 3D 堆叠有源中介层组织，把计算芯粒与缓存和互连分离。

MI300A APU 还有一些关键差异，使其相较早期 EHP 概念获得显著优势和改进。也许最重要的是，混合键合 3D 堆叠与基于微凸点的 2.5D 中介层相结合，使所有计算芯粒、HBM 和 I/O 之间都能采用带宽高得多、能效更好的数据传输底层结构。EHPv4 因 GPU 裸片间距离较大而效率不佳；即使在 EHPv3 中，有源中介层之间基于有机基板的链路也会面临带宽和功耗挑战。

EHPv3 需要两种不同的有源中介层，会增加设计成本。MI300A 实际上只需一套 IOD 设计，就能混合搭配不同计算芯粒。相较 EHPv4 尝试复用 EPYC 处理器 IOD 的做法，为 MI300A 设计新的 IOD 是更好的选择。硅设计复用通常是可取的，但只有与目标应用场景契合时才有效；对于 EHPv4，EHP 的需求与 EPYC 服务器产品的设计要求差距过大。

VI. 编程模型与软件

软件可编程性是 EHP 的重要组成部分，也是 MI300A 设计中的首要考虑之一。为简化 MI300A 软件开发，我们建立了相应机制，将多个 XCD 呈现为一个统一的 GPU，并让 CPU 与 GPU 共享统一的 HBM 内存。

A. 统一的多芯粒加速器

EHP 的目标之一，是满足用户对每一代 GPU 都能让内核获得更高性能期待。然而，我们发现 EHPv4 并不适合让单个内核在两个 GPU 芯粒上执行，因为芯粒间连接带宽低于 HBM 带宽。出于类似原因，AMD Instinct™ MI250X 将每个 GCD 呈现为独立加速器。但随着设计中的芯粒数量增加，把每个芯粒都作为单独加速器暴露给开发者，就不那么有吸引力了。

用户态软件与 MI300A 之间用于启动内核的接口，是位于用户态可见内存中的队列，软件可以向其中填入描述内核的数据包。队列由异构系统架构（HSA）标准 [18] 定义，提交到队列中的数据包遵循 HSA 的架构化队列语言（AQL）格式。更底层的数据包格式会描述应向哪些硬件寄存器写入什么值，才能让某个 GPU 启动内核；相比之下，AQL 包描述更高层的目标，例如“启动内核 X，共 Y 个工作组，每组 Z 个线程”。

如第 IV.B 节所述，每个 MI300A XCD 都包含将内核分派到该 XCD 所需的硬件。这些异步计算引擎（ACE）负责连接用户态队列、读取 AQL 包、解码这些包，并适当配置微架构资源以开始内核分派。随后，ACE 硬件在 XCD 的计算单元中为工作组寻找可用空间，初始化波前寄存器状态，并设置程序计数器，使其开始执行。最后，ACE 硬件负责检测一个内核的所有工作组何时完成，并向软件发出完成指示。

MI250X 等 AMD GPU 中的 ACE 也以类似方式工作，但 MI300A 还允许每个 XCD 上的内核调度硬件与其他 XCD 协作，向用户态软件呈现一个包含多个 XCD 的分区。由于 AQL 包描述的是“启动一个内核”这样的高层请求，每个 XCD 上的 ACE 可以协同处理。

例如，当分派包被提交到队列时，一个分区中每个 XCD 上的 ACE 都会读取该 AQL 包 ①。所有这些处理器都会解码数据包，并配置自身的局部微架构，以启动所请求工作组中的一个子集 ②。ACE 知道分区内有多少个 XCD，因此也知道自身 XCD 只负责执行该内核全部工作组的一部分。哪些工作组被调度到哪个 XCD 可以配置，以便在不同因素间权衡，例如让工作组通过 XCD 的 L2 缓存复用数据，或让尽可能多的 XCD 开始工作以最大化内存带宽。决定启动哪些工作组后，XCD 内部便按常规方式创建工作组。

在数据包处理的若干阶段，各 XCD 的 ACE 可能需要彼此同步 ③。例如，所有 XCD 都必须表明：自身负责的分派波前子集已经完成，并且这些波前的写入在相应一致性作用域内可见。随后，一个指定的 XCD 才能发送表示内核完成的信号 ④。MI300A 的 Infinity Fabric 包含高优先级通信通道，以支持分区内各 XCD 的 ACE 之间进行高效通信与同步。

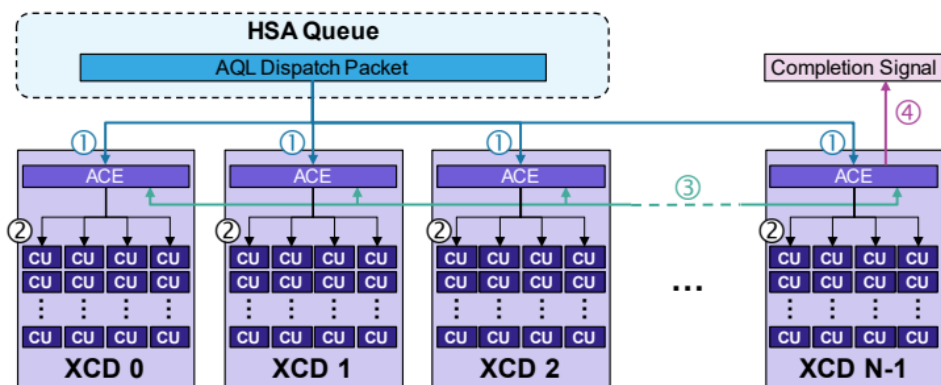


图 13. 跨多个 XCD 的内核分派与完成流程。

图内中文对照：HSA Queue：HSA 队列；AQL Dispatch Packet：AQL 分派包；Completion Signal：完成信号。① 各 XCD 的 ACE 读取分派包；② 启动各自负责的工作组；③ ACE 之间协作与同步；④ 指定 XCD 发出完成信号。

这一协作协议能够从一个软件可见队列接收一次内核分派，并将工作组分布到多个 XCD 上，使多芯粒分区表现为一个逻辑 GPU。它与 MI300A 的统一内存相配合：整个 HBM 内存空间对单个进程可见。我们没有使用单独的调度芯粒，而是在各芯粒上设置调度器，以减少芯粒间布线需求，并在增加芯粒时提高工作组调度吞吐量。这也使 MI300A 能呈现包含不同数量芯粒的分区，见第 VIII 节。

将 MI300A 设备呈现为单个分区，可以帮助现有软件轻松利用整个设备。它支持 HPC 中常见的多种软件框架，包括 OpenMP®、RAJA [6] 和 Kokkos [10]，也支持 PyTorch、TensorFlow、JAX [10]、Triton [38] 等常用 AI / ML 框架。AMD ROCm™ 开放计算平台 [3] 提供开发者熟悉的优化库，例如 rocBLAS、rocDNN，以便使用常见计算任务的优化实现。

B. 统一内存

MI300A 为 CPU 核心和 GPU CU 呈现单一的统一内存，简化了编程模型。统一内存可以消除冗余数据复制，去掉主机与设备内存空间的区分，并支持处理单元间的细粒度共享。有些平台向软件呈现统一内存的外观，例如通过页面迁移，在 CPU 的 DDR 和 GPU 的 HBM 之间透明复制数据；MI300A 则让实际物理内存组织与程序员所见的视图一致，避免了此类数据传输开销。

图 14 展示了代码简化示例。在图 14(a) 的纯 CPU 代码中，程序先分配内存，例如调用 malloc，然后初始化，再对数据进行计算。CPU 和 GPU 内存分离时，代码更复杂，如图 14(b) 所示。首先必须分别为 CPU 和 GPU 分配内存空间，调用 malloc 和 hipMalloc。随后 CPU 初始化内存，但还必须用 hipMemcpy 将数据从 CPU 内存空间复制到 GPU 内存缓冲区。之后 CPU 才能在 GPU 上启动内核，并等待 GPU 完成，例如调用 hipDeviceSynchronize。CPU 无法立即继续内核之后的自身计算，而要先通过另一次 hipMemcpy 将结果从 GPU 内存复制回 CPU 一侧，然后才能完成其余操作。

采用 APU 编程模型，例如图 14(c) 中的 MI300A 时，程序员可以省去 CPU 与 GPU 内存分离所要求的管理步骤。APU 代码无需分配 GPU 内存，因此没有 hipMalloc；由于与纯 CPU 情况一样，数据只有一份，也无需搬来搬去，因此没有 hipMemcpy。得益于 APU 的统一内存，包括 CPU 与 GPU 之间的缓存一致性，CPU 初始化完数据后就可以立即启动 GPU 内核。CPU 仍须与 GPU 正确同步，之后便可继续内核后的计算，同样无需额外的数据复制。

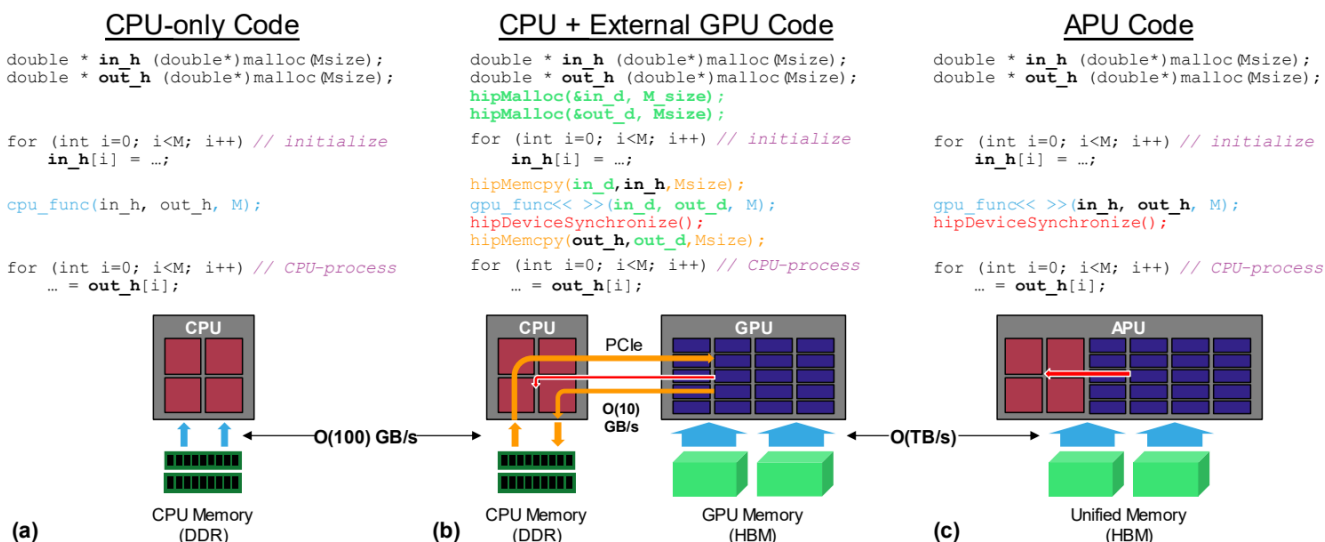


图 14. 三种情形下的示例代码、数据传输与同步：(a) 仅 CPU；(b) CPU 与内存空间独立的分立 / 外部 GPU；(c) 具有统一内存的 APU。

图内中文对照：CPU-only Code：仅 CPU 的代码；CPU + External GPU Code：CPU 与外部 GPU 的代码；APU Code：APU 代码；CPU Memory：CPU 内存；GPU Memory：GPU 内存；Unified Memory：统一内存。代码注释 initialize：初始化；CPU-process：CPU 处理。O(100) GB/s、O(10) GB/s、O(TB/s) 分别表示数百 GB/s、数十 GB/s 和 TB/s 量级。代码与函数名保持原样。

除了可编程性收益，MI300A 等 APU 还可以通过优化数据传输来提高性能。对于分立 GPU，主机 CPU 与外部 GPU 之间的带宽通常受 PCIe® 接口限制，一般只有数十 GB/s。图 14(b) 下半部分展示了分立 GPU 场景：CPU 可以以数百 GB/s 从 DDR 内存读取数据，但随后受限于 PCIe。图 14(c) 的 APU 场景中，CPU 尽其所能地向 HBM 写入数据，GPU 随后就能立即按 HBM 的速度处理这些数据。

APU 编程模型还提供了进一步优化 CPU 与 GPU 协作的新机会。图 15(a) 的伪代码展示了如何利用统一内存，让 GPU 与 CPU 计算重叠，并去除粗粒度的内核级同步。示例增加了一个完成标志数组，每个输出元素对应一个标志，使 GPU 能够在生成输出数据时设置相应标志，如图 15(b) 所示。借助 APU 的缓存一致性内存系统，CPU 一侧可以通过自旋循环等待。GPU 每生成一个数据元素，CPU 就能立即开始相应的后处理，从而提高计算并发度。

Optimized APU Code on MI300A

```
double * in_h (double*)malloc(Msize);
double * out_h (double*)malloc(Msize);
int* flag = (int*)malloc(Isize);

for (int i=0; i<M; i++) { // initialize
    in_h[i] = ...;
    flag[i] = 0;
}

gpu_func<<>>(in_h, out_h, M);
hipDeviceSynchronize();

for (int j=0; j<Isize; j++) {
    while(atomic_load(&flag[j]) == 0) {}
    post_process(out_h[j]); // CPU-process
}
```

(a)

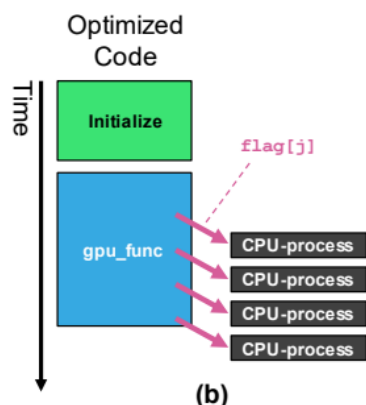


图 15. (a) 对 GPU 与 CPU 执行进行细粒度解耦的示例代码；(b) 展示 GPU 与 CPU 执行重叠的时间线；(c) 原始代码的时间线。

图内中文对照：Optimized APU Code on MI300A：MI300A 上优化后的 APU 代码；Optimized Code：优化后的代码；Time：时间；Initialize：初始化；CPU-process：CPU 处理；flag[j]：第 j 个完成标志。代码注释 initialize 译为“初始化”，CPU-process 译为“CPU 处理”。原图中 hipDeviceSynchronize() 被划去。译注：本地原件图题提到 (c)，但实际图像只包含 (a) 与 (b)，此处保留原图和原题。

将设备呈现为具有统一内存空间的单个加速器，还为自动在 APU 上加速应用提供了机会，因为 CPU 核心与 GPU CU 始终都能通过封装内 HBM 访问数据。这意味着软件大体上无需考虑跨链路访问非本地数据的开销。因此，BLAS、LAPACK 等标准库 API 可以同时链接到 CPU 库和 GPU 库。通用库调用先进入一个轻量适配库，再根据问题规模等简单启发式规则，将任务分派给 CPU 或 GPU 处理单元。这样，原本可能只在 CPU 上运行、但频繁调用标准库的代码，也可以卸载到 APU，而无需显式重构代码。

VII. MI300X

如引言所述，业界正在经历 ML 和生成式 AI 工作负载计算需求的爆发式增长。许多客户希望获得既有巨大计算吞吐量，又有高带宽、大容量内存系统的方案。大语言模型（LLM）正是这种要求很高的现代工作负载的典型例子。有些细分市场采用 AI 专用加速器，但许多用户更希望使用可编程性更好的通用 GPU 架构，例如我们的 CDNA XCD 所支持的架构。

MI300A 的硅构件提供了一个模块化芯粒平台，可以在 IOD 上堆叠不同的计算芯粒。AMD 通过复用 MI300A 的硅组件，还设计了 AMD Instinct MI300X 加速器 [33]。MI300X 的结构与 MI300A 类似，但将三个 CCD 替换为一对 XCD，形成仅含加速器的模块，如图 16 所示。八个 XCD 合计提供 304 个 CU，单位体积浮点运算性能（FLOPS/mm³）高于 MI300A。MI300X 还采用 12 层 HBM 堆叠，每个堆叠 24 GB，总内存容量为 192 GB。

MI300X 尤其适合 LLM：提示词处理阶段需要高计算吞吐量，词元生成阶段通常受内存带宽限制，而现代 LLM 还需要更大的总内存容量来容纳规模更大的模型。

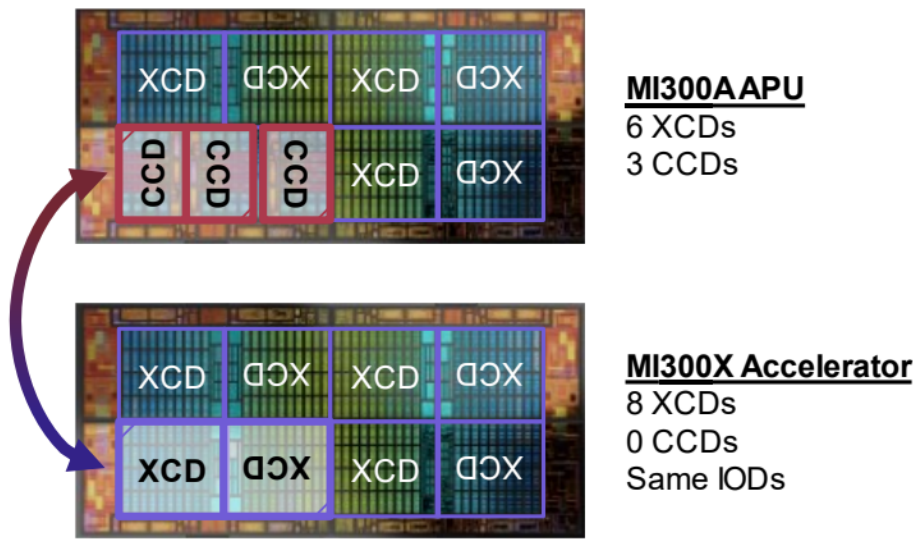


图 16. 通过模块化替换，将 MI300A 的 CCD 换为 XCD，构成 MI300X 加速器。

图内中文对照：MI300A APU：6 个 XCD、3 个 CCD；MI300X Accelerator：MI300X 加速器，8 个 XCD、0 个 CCD；Same IODs：使用相同的 IOD。

我们的模块化设计方法，将已经取得成功的芯粒策略 [27] 推进到了新的层次。以往的芯粒方案是在不同产品线中复用同一类型芯粒，例如 EPYC 和 Ryzen 处理器；这里则采用新的芯粒模块化方式，可以按需替换不同类型的芯粒。

AMD 只需设计 XCD 和 IOD，再通过 IOD 镜像以及对现有 CCD 的适配，就能够组合这些组件，形成 MI300A 和 MI300X 两类产品，而不牺牲任何一方。MI300 呈现了一种新的模块化架构，也切实体现了异构集成方法：在同一个封装内结合芯粒、不同工艺节点、可混合搭配的芯粒接口、2.5D 硅中介层、3D HBM 和 3D 混合键合。

VIII. 平台架构与可扩展性

芯粒方案提供了设计灵活性，MI300 还具有其他能力，让用户可以灵活部署。第一类选项允许 MI300A 和 MI300X 支持不同分区配置，以用于多用户部署。

对于 MI300A，六个 XCD 可以作为单个计算设备使用，也可以划分成三个独立分区，如图 17(a) 所示。两种分区模式下，整个 HBM 地址空间都统一交错分布，每个插槽实现一个非统一内存访问（NUMA）域，即 NPS1。只含 XCD 的 MI300X 支持更多分区选项，如图 17(b) 所示。XCD 可以按二的幂次划分，从单个统一分区一直到八个独立分区，每个分区一个 XCD。内存系统既支持单个统一 NUMA 域（NPS1），也支持把内存空间分成每插槽四个 NUMA 域（NPS4）。这种方案适合 PCIe® 单根 I/O 虚拟化（SR-IOV），每个 PCIe 虚拟功能（VF）都可以映射到独立分区。

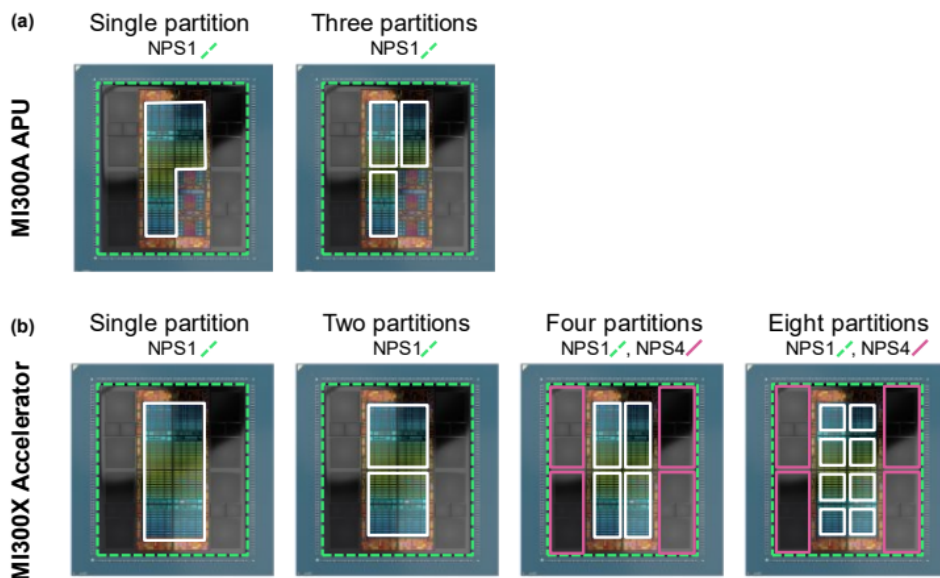


图 17. (a) MI300A APU 与 (b) MI300X 加速器的计算和内存分区模式。

图内中文对照：Single / Two / Three / Four / Eight partition(s)：单个 / 两个 / 三个 / 四个 / 八个分区。MI300A 的单分区与三分区均采用 NPS1；MI300X 的单分区和双分区采用 NPS1，四分区与八分区可采用 NPS1 或 NPS4。

MI300 架构还支持 I/O 配置，以便将多个 MI300 模块组织为多种可扩展节点拓扑。每个 IOD 提供两个 x16 I/O 接口。其中一条 x16 链路专用于 Infinity Fabric 连接，另一条既可用作 Infinity Fabric 链路，也可用作 PCIe 第五代接口。每个 MI300 插槽合计具有八条 x16 链路，其中四条可在 Infinity Fabric 与 PCIe 之间选择。每条 x16 链路的理论峰值双向带宽为 128 GB/s，每方向 64 GB/s；每插槽总计为 1024 GB/s。

图 18(a) 展示了采用四个 MI300A APU 的节点架构示例。每个 MI300A 都能通过加载和存储，直接访问四个模块中的全部 HBM，即采用平坦物理地址空间。在这个例子中，每个 MI300A 插槽使用八条 x16 I/O 链路中的六条，与其他 MI300A 模块建立全连接拓扑的缓存一致性 Infinity Fabric 链路，也就是每对 MI300A APU 之间有两 x16 链路。其余 I/O 接口可供网络接口、存储等使用。

图 18(b) 展示了另一种系统拓扑：八个 MI300X 加速器模块作为 PCIe 设备连接到 EPYC CPU 主机。此时，每个 MI300X 模块使用八条 x16 I/O 链路中的七条，在八个 MI300X 加速器之间构成全连接 IF 拓扑。最后一条 PCIe 链路连接回 CPU 主机，根据具体节点架构，也可能经过 PCIe 交换机。需要注意的是，在这两种可扩展系统拓扑中，MI300A 与 MI300X 使用完全相同的 IOD 设计，只是利用接口可配置性形成了非常不同的整体系统。

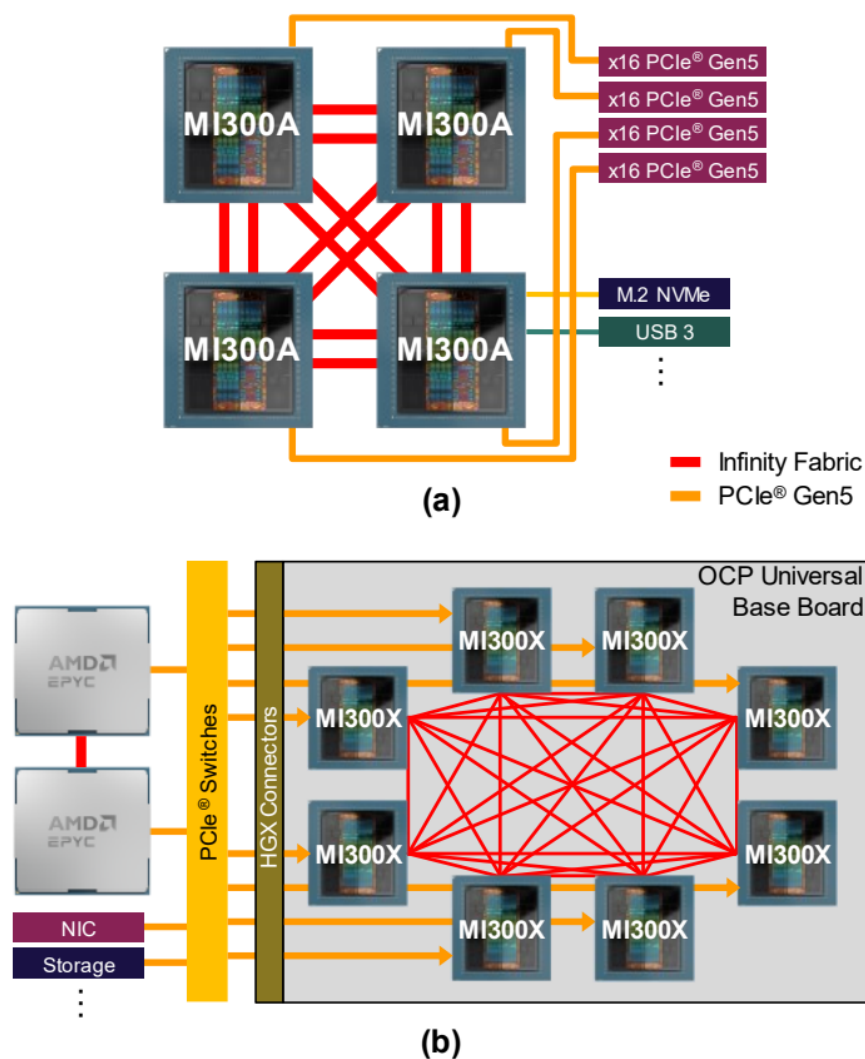


图 18. (a) MI300A APU 与 (b) MI300X 加速器的节点架构配置示例。

图中中文对照：PCIe Gen5：PCIe 第五代；PCIe Switches：PCIe 交换机；HGX Connectors：HGX 连接器；OCP Universal Base Board：OCP 通用底板；NIC：网络接口卡；Storage：存储。红色链路为 Infinity Fabric，橙色链路为 PCIe 第五代；M.2 NVMe 和 USB 3 保留接口名称。

IX. 结果

这篇产业分会论文的重点并非最终性能数字，而是向计算机体系结构研究群体分享我们的见解、经验，以及从早期 EHP 研究概念演进为 APU 和加速器产品系列的过程。这些产品面向 HPC 和 AI 市场，能够大批量制造，性能很高，设计相当激进，却仍可切实生产。本节只提供少量结果，帮助读者了解 MI300A 和 MI300X 的收益。

MI300A APU 与 MI300X 加速器相较 AMD Instinct MI250X 加速器都带来了代际性能提升。图 19 将 MI250X、MI300A 和 MI300X 的多个重要指标并列比较。各类峰值计算吞吐量均有所提升，峰值内存带宽也提高了 70%，I/O（网络）带宽翻倍，以支持 HPC 和 AI / ML 的横向扩展。MI300X 的总内存容量还增加了 50%，以更好地服务 LLM。

		AMD Instinct MI250X (Up to)	AMD Instinct MI300A (Up to)	AMD Instinct MI300X (Up to)
Hardware Specs	Memory Capacity	128 GB HBM2e	128 GB HBM3	192 GB HBM3
	Memory Bandwidth (Peak Theoretical)	~3.2 TB/s	~5.3 TB/s	~5.3 TB/s
	Scale-Out (Back-end) Network Bandwidth	200 Gb/s Ethernet/IB	400 Gb/s Ethernet/IB	400 Gb/s Ethernet/IB
	Max TDP/TBP	560 W	760 W	760 W
	Heterogeneous Integration	2D IF, 2.5D EFB, 3D μbump HBM	2.5D passive silicon interposer, 3D HB active interposer and multiple chiplets, 3D μbump HBM	
HPC Peak Perf. (Peak)	FP64 Vector (TFLOPS)	47.9	61.3	81.7
	FP32 Vector (TFLOPS)	47.9	122.6	163.4
	FP64 Matrix (TFLOPS)	95.7	122.6	163.4
	FP32 Matrix (TFLOPS)	95.7	122.6	163.4
AI Peak Perf. (Peak)	TF32* [TF32 Sparsity] (Matrix)	Not Supported	490.3 [980.6]	653.7 [1307.4]
	FP16 [FP16 Sparsity] (TFLOPS)	383.0 [Not Supported]	980.6 [1961.2]	1307.4 [2614.9]
	BFLOAT16 [BF16 Sparsity] (TFLOPS)	383.0 [Not Supported]	980.6 [1961.2]	1307.4 [2614.9]
	FP8* [FP8 Sparsity] (TFLOPS)	Not Supported	1961.2 [3922.3]	2614.9 [5229.8]
	INT8 [INT8 Sparsity] (TOPS)	383.0 [Not Supported]	1961.2 [3922.3]	2614.9 [5229.8]

图 19. MI300A 和 MI300X 相较 MI250X 的代际提升。

图内中文对照：图中数据为最高规格（Up to）。Hardware Specs：硬件规格；HPC Peak Perf.：HPC 峰值性能；AI Peak Perf.：AI 峰值性能；Memory Capacity：内存容量；Memory Bandwidth (Peak Theoretical)：理论峰值内存带宽；Scale-Out (Back-end) Network Bandwidth：横向扩展（后端）网络带宽；Max TDP/TBP：最大 TDP / TBP；Heterogeneous Integration：异构集成；Vector：向量；Matrix：矩阵；Sparsity：稀疏；Not Supported：不支持。下表提供完整中文对照，方括号为稀疏运算值。

指标（最高规格）	MI250X	MI300A	MI300X
内存容量	128 GB HBM2e	128 GB HBM3	192 GB HBM3
理论峰值内存带宽	约 3.2 TB/s	约 5.3 TB/s	约 5.3 TB/s
横向扩展后端网络带宽	200 Gb/s Ethernet/IB	400 Gb/s Ethernet/IB	400 Gb/s Ethernet/IB
最大 TDP / TBP	560 W	760 W	760 W
异构集成	2D IF、2.5D EFB、3D 微凸点 HBM	2.5D 无源硅中介层、3D 混合键合有源中介层与多个芯粒、3D 微凸点 HBM	同 MI300A
FP64 向量 (TFLOPS)	47.9	61.3	81.7
FP32 向量 (TFLOPS)	47.9	122.6	163.4
FP64 矩阵 (TFLOPS)	95.7	122.6	163.4
FP32 矩阵 (TFLOPS)	95.7	122.6	163.4
TF32* 矩阵 [稀疏]	不支持	490.3 [980.6]	653.7 [1307.4]
FP16 [稀疏] (TFLOPS)	383.0 [不支持]	980.6 [1961.2]	1307.4 [2614.9]
BFLOAT16 [稀疏] (TFLOPS)	383.0 [不支持]	980.6 [1961.2]	1307.4 [2614.9]
FP8* [稀疏] (TFLOPS)	不支持	1961.2 [3922.3]	2614.9 [5229.8]
INT8 [稀疏] (TOPS)	383.0 [不支持]	1961.2 [3922.3]	2614.9 [5229.8]

Ethernet/IB：以太网 / InfiniBand；TFLOPS：每秒万亿次浮点运算；TOPS：每秒万亿次运算。星号沿用原图；本地原件未附对应星号说明。

图 20 展示了 MI300A 相较上一代 MI250X 加速器在若干 HPC 工作负载上的性能加速比。MI300A 结果在 MI300A 初始调试参考平台上测得，配置为 128 GB HBM3、550 W 热设计功耗（TDP），运行 Ubuntu® Linux 和 ROCm 6.0；MI250X 结果的配置为 128 GB HBM2e、560 W 板卡总功耗（TBP），使用 Ubuntu

Linux 和 ROCm 5.4.3。全部测试都仅使用一个 APU 或一个 GPU。

原文脚注：TDP = Thermal Design Power（热设计功耗），TBP = Total Board Power（板卡总功耗）。计入稳压损耗后，MI300A 的 TBP 约为 670 W。

对于 GROMACS、N 体内核 [16] 和 HPCG 基准 [17]，MI300A 获得更高性能，原因分别是更高的计算吞吐量（GROMACS、N 体）以及 HBM3 相较 MI250X 所用 HBM2e 的更高内存带宽（HPCG）。OpenFOAM® 是计算流体力学工作负载 [15]，性能达到原来的 2.75 倍。该工作负载使用 HPC Motorbike 测试 / 输入，与 APU 计算范式十分契合，因为它：（1）计算密集；（2）需要高内存带宽；（3）在分立 GPU 实现中往往有大量 CPU 与 GPU 之间的数据传输。MI300A 丰富的计算资源和八个 HBM 堆叠满足了前两个需求。APU 的统一内存架构则解决了数据传输问题，因为在可能的情况下，最快的数据传输方式就是根本不移动数据。

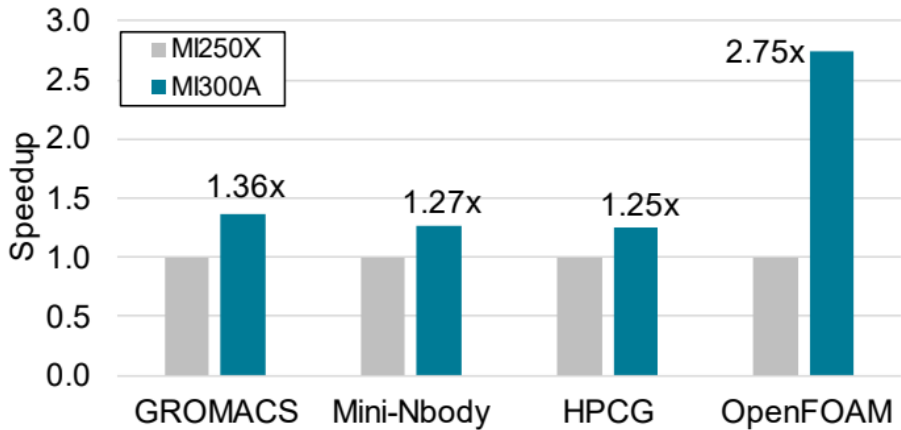


图 20. AMD MI300A APU 相较 AMD MI250X 加速器在 HPC 工作负载上的实测加速比。

图内中文对照：Speedup：加速比。MI250X 归一化为 1；MI300A 在 GROMACS、Mini-Nbody、HPCG 和 OpenFOAM 上分别为 1.36 倍、1.27 倍、1.25 倍和 2.75 倍。

虽然本文重点是面向百亿亿次级计算的 MI300A APU，我们也展示了一些 MI300X 加速器的结果。图 21 给出了 700 亿参数 Llama-2 模型的推理性能 [39]。这些实验采用批量大小 1、2048 个输入词元和 128 个输出词元。完整平台配置细节可在线查阅 [6]。

左侧第一组结果，分别在基准 GPU 与 MI300X 上使用 vLLM 推理与服务库 [21]，直接比较硬件能力，展示推理延迟中位数的改善。该实验测得 MI300X 的推理延迟性能改善超过两倍。第二组结果在基准 GPU 上使用专为其优化的 TensorRT-LLM 库，即使如此，MI300X 仍带来约 30% 的改善。最后一组结果考虑了基准 GPU 使用 FP8 数值格式的情况，这相当于让基准 GPU 的峰值计算和内存吞吐量翻倍。由于当时 vLLM 库尚不支持 FP8，MI300X 结果仍采用 FP16 格式。尽管数值格式不同，在测量绝对延迟时，MI300X 仍表现出性能优势。

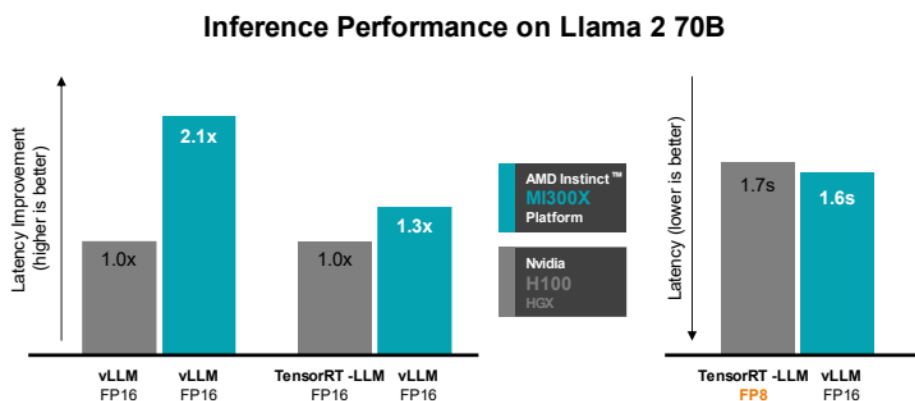


图 21. Llama-2 70B 推理延迟性能（中位数），批量大小为 1，输入 2048 个词元，输出 128 个词元。

图内中文对照：Inference Performance on Llama 2 70B: Llama 2 70B 推理性能；Latency Improvement (higher is better): 延迟性能改善倍数（越高越好）；Latency (lower is better): 延迟（越低越好）。灰色为 NVIDIA H100 HGX，蓝绿色为 AMD Instinct MI300X 平台。同用 vLLM FP16 时为 1.0 与 2.1 倍；基准用 TensorRT-LLM FP16、MI300X 用 vLLM FP16 时为 1.0 与 1.3 倍；基准用 TensorRT-LLM FP8 为 1.7 秒，MI300X 用 vLLM FP16 为 1.6 秒。

X. 结论

虽然我们面向 HPC 的 APU 愿景在十多年前就已萌芽，但有些想法无论在技术上多么有吸引力，都不能在技术、商业及其他条件尚未准备好时仓促投入生产。基于 MI250X 的 Frontier 节点架构，与仅相隔几年推出的 MI300A APU 设计之间的对比，说明路线图和产品计划必须根据具体情况谨慎调整。第一代百亿亿次级超级计算机没有实现 APU，并不意味着 HPC APU 的想法在技术上有问题，只是时机尚未成熟。随着技术成熟、市场变化和客户需求演进，AMD 推出高性能 APU 的时机终于到来。

参考文献

每条先列中文题名，再保留原文作者、题名、出版信息及链接，以便检索。

[1] AMD Instinct™ MI200 系列加速器

Advanced Micro Devices, Inc., "AMD Instinct™ MI200 Series Accelerator," 2022, <https://www.amd.com/system/files/documents/amd-instinct-mi200-datasheet.pdf>.

[2] AMD CDNA™ 2 架构介绍

Advanced Micro Devices, Inc., "Introducing AMD CDNA™ 2 Architecture," 2021, <https://www.amd.com/system/files/documents/amd-cdna-whitepaper.pdf>

[3] AMD ROCm: 面向 GPU 计算的开放软件平台

Advanced Micro Devices, Inc., "AMD ROCm Open Software Platform for GPU Compute," <http://www.amd.com/ROCm>, 2022.

[4] 第四代 AMD EPYC™ 处理器架构

Advanced Micro Devices, Inc., "4th-gen AMD EPYC™ Processor Architecture," <https://www.amd.com/system/files/documents/4th-gen-epyc-processor-architecture-white-paper.pdf>

[5] AMD CDNA™ 3 架构

Advanced Micro Devices, Inc., "AMD CDNA™ 3 Architecture," <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>

[6] AMD Instinct MI300X 的竞争性能声明与行业领先的推理性能

Advanced Micro Devices, Inc., “Competitive performance claims and industry leading Inference performance on AMD Instinct MI300X,” <https://community.amd.com/t5/instinct-accelerators/competitive-performance-claims-and-industry-leading-inference/ba-p/652304>, December 2023

[7] 项目：百亿亿次级计算研究与开发

Argonne National Laboratory, Request for Information (RFI) No. 1-KD73-I-31583-00, “Project: Exascale Research and Development,” July 2011.

[8] RAJA：为大规模科学应用提供可移植的性能

D. Beckingsale, J. Burmark, R. Hornung, H. Jones, W. Killian, A. Kunen, O. Pearce, P. Robinson, B. Ryuji, T. Scogland, “RAJA: Portable Performance for Large-Scale Scientific Applications,” Int’l Workshop on Performance, Portability and Productivity in HPC, 2019.

[9] 回顾登纳德 MOSFET 缩放论文的三十年

M. Bohr, “A 30 Year Retrospective on Dennard’s MOSFET Scaling Paper,” IEEE Solid-State Circuits Society Newsletter, vol. 12(1), 2007.

[10] JAX: Python+NumPy 程序的可组合变换

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, “JAX: Composable Transformations of Python+NumPy programs,” <http://github.com/google/jax>, 2018

[11] 使用 TSV 技术与微凸点互连实现 DRAM 的 3D 堆叠

K.-W. Chung, S. Shih, S.-T. Lu, T.-H. Chen, C.-T. Chen, J. Ho, J.-J. Chen, J.-P. Lin, “3D Stacking DRAM using TSV Technology and Microbump Interconnect,” in the International Microsystems Packaging Assembly and Circuits Technology Conference, October 2010.

[12] 2.5D 系统集成方案的互连特性

Y. Deng and W. Maly. “Interconnect Characteristics of 2.5-D System Integration Scheme,” Proceedings of the International Symposium on Physical Design, pages 171–175, April 2001.

[13] Kokkos：通过多态内存访问模式实现众核性能可移植性

H. Carter Edwards, Christian R. Trott, and Daniel Sunderland. “Kokkos: Enabling manycore performance portability through polymorphic memory access patterns.” Journal of parallel and distributed computing 74.12 (2014): 3202-3216.

[14] AMD 下一代 “Zen 3” 核心

M. Evers, L. Barnes, M. Clark, “The AMD Next-Generation ‘Zen 3’ Core,” in IEEE Micro, Vol. 42(3), May-June 2022.

[15] OpenFOAM v11 用户指南

C. Greenshields, “OpenFOAM v11 User Guide,” The OpenFOAM Foundation, 2023, <https://doc.cfd.direct/openfoam/user-guide-v11>

[16] mini-nbody：一个简单的 N 体计算程序

M. Harris, “mini-nbody: A simple N-body Code,” <https://github.com/harrism/mini-nbody>

[17] HPCG 技术规范

M. A. Heroux, J. Dongarra, P. Luszczek, “HPCG Technical Specification,” Sandia National Laboratories Technical Report, SAND2013-8752, October 2013.

[18] HSA 平台系统架构规范，版本 1.2

HSA Foundation, “HSA Platform System Architecture Specification Version 1.2,” May 2018.

[19] 异构集成路线图（2021 年版）

IEEE Electronics Packaging Society, “Heterogeneous Integration Roadmap (2021 Edition), <http://eps.ieee.org/hir>

[20] 支持基于中介层的多核处理器拆分

A. Kannan, N. Enright Jerger, G. H. Loh, "Enabling Interposer-based Disintegration of Multi-core Processors," International Symposium on Microarchitecture (MICRO), December 2015.

[21] vLLM: 利用 PagedAttention 提供简便、快速且低成本的 LLM 服务

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joey Gonzalez, Hao Zhang, Ion Stoica, "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention," <https://blog.vllm.ai/2023/06/20/vllm.html>

[22] AMD 百亿亿次级计算历程的研究回顾

G. Loh, M. Schulte, M. Ignatowski, V. Adhinarayanan, S. Aga, D. Aguren, V. Agrawal, A. M. Aji, J. Alsop, P. Bauman, B. M. Beckmann, M. V. Beigi, S. Blagodurov, T. Boraten, M. Boyer, W. Brantley, N. Chalmers, S. Chen, K. Cheng, M. Chu, D. Cownie, N. Curtis, J. del Pino, N. Duong, A. Dutu, Y. Eckert, C. Erb, C. Freitag, J. Greathouse, S. Gurumurthi, A. Gutierrez, K. Hamidouche, S. Hossamani, W. Huang, M. Islam, N. Jayasena, J. Kalamatianos, O. Kayiran, J. Kotra, A. Lee, D. Lowell, N. Madan, A. Majumdar, N. Malaya, S. Manne, S. Mashimo, D. McDougall, E. Mednick, M. Mishkin, M. Nutter, I. Paul, M. Poremba, B. Potter, K. Punniyamurthy, S. Puthoor, S. Raasch, K. Rao, G. Rodgers, M. Scrbak, M. Seyedzadeh, J. Slice, V. Sridharan, R. van Oostrum, E. van Tassell, A. Vishnu, S. Wasmundt, M. Wilkening, N. Wolfe, M. Wyse, A. Yalavarti, D. Yudanov, "A Research Retrospective on AMD's Exascale Computing Journey," in the International Symposium on Computer Architecture, June 2023.

[23] 为百亿亿次级计算准备应用的经验

N. Malaya, B. Messer, J. Glenski, A. Georgiadou, J. Lietz, K. Gottiparthi, M. Day, J. Chen, J. Rood, L. Esclapez, J. White III, G. R. Jansen, N. Curtis, S. Nichols, J. Kurzak, N. Chalmers, C. Freitag, P. Bauman, A. Fanfarillo, R. D. Budiardja, T. Papatheodore, N. Frontiere, D. McDougall, M. Norman, S. Sreepathi, P. Roth, D. Bykov, N. Wolfe, P. Mullaney, M. Eisenbach, M. T. Henry De Frahan, W. Joubert, "Experiences readying applications for Exascale," in the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23), November 2023.

[24] 评估 LUMI 超级计算机的 GPU 编程模型

G. S. Markomanolis, A. Alpay, J. Young, M. Klemm, N. Malaya, A. Esposito, J. Heikonen, S. Bastrakov, A. Debus, T. Kluge, K. Steiniger, J. Stephan, R. Widera, M. Bussmann, "Evaluating GPU Programming Models for the LUMI Supercomputer," in Supercomputing Frontiers: 7th Asian Conference, March 2022.

[25] 在集成电路上容纳更多元件

G. E. Moore, "Cramming More Components onto Integrated Circuits," in Electronics, Vol. 38, No. 8, April 1965.

[26] "Zen 4": AMD 5 nm、5.7 GHz x86-64 微处理器核心

B. Munger, K. Wilcox, J. Sniderman, C. Tung, B. Johnson, R. Schreiber, C. Henrion, K. Gillespie, T. Burd, H. Fair, D. Johnson, J. White, S. McLelland, S. Bakke, J. Olson, R. McCracken, M. Pickett, A. Horiuchi, H. Nguyen, T. H. Jackson, "'Zen 4': The AMD 5nm 5.7GHz x86-64 Microprocessor Core," in the International Solid-State Circuits Conference, February 2023.

[27] 为 AMD EPYC™ 和 Ryzen™ 处理器系列开拓芯粒技术与设计

S. Naffziger, N. Beck, T. Burd, K. Lepak, G. H. Loh, M. Subramony, S. White, "Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families," in the International Symposium on Computer Architecture (ISCA), June 2021.

[28] 不退缩: DOE、Cray 和 AMD 将为 ORNL 带来百亿亿次级计算

Oak Ridge National Laboratory, "No Scaling Back: DOE, Cray, AMD to Bring Exascale to ORNL," May 2019, <https://www.olcf.ornl.gov/2019/05/07/no-scaling-back-doe-cray-amd-to-bring-exascale-to-ornl/>

[29] Frontier 超级计算机首次亮相即成为世界最快, 突破百亿亿次级计算门槛

Oak Ridge National Laboratory, "Frontier Supercomputer Debuts as World's Fastest, Breaking Exascale Barrier," <https://www.ornl.gov/news/frontier-supercomputer-debuts-worlds-fastest-breaking-exascale-barrier>

[30] Frontier 用户指南

Oak Ridge National Laboratory, "Frontier User Guide," 2022, https://docs.olcf.ornl.gov/systems/frontier_user_guide.html

[31] 通过异构计算实现百亿亿次级计算能力

M. J. Schulte, M. Ignatowski, G. H. Loh, B. M. Beckmann, W. C. Brantley, S. Gurumurthi, N. Jayasena, I. Paul, S. K. Reinhardt, G. Rodgers, "Achieving Exascale Capabilities through Heterogeneous Computing," IEEE Micro, July-August 2015.

[32] AMD Instinct™ MI200 系列加速器与节点架构

A. Smith, N. James, “AMD Instinct™ MI200 Series Accelerator and Node Architectures,” in Hot Chips, August 2022.

[33] AMD Instinct™ MI300 系列模块化芯粒封装：面向百亿亿次级系统的 HPC 与 AI 加速器

A. Smith, E. Chapman, C. Patel, R. Swaminathan, J. J. Wu, T. Huang, W. Jung, A. Kaganov, H. McIntyre, R. Mangaser, “AMD Instinct™ MI300 Series Modular Chiplet Package - HPC and AI Accelerator for Exa-Class Systems,” in the International Solid-State Circuits Conference, February 2024.

[34] 面向科学的 AI：美国能源部科学人工智能公众研讨会报告

R. Stevens, V. Taylor, J. Nichols, A. B. Maccabe, K. Yelick, D. Brown, “AI for Science: Report on the Department of Energy (DOE) Town Halls on Artificial Intelligence (AI) for Science,” ANL-20/17 158802. February 2020.

[35] 案例研究：采用 3D 封装构建的 AMD 产品（教程）

R. Swaminathan, “Case Study: AMD Products Built with 3D Packaging (Tutorial),” in Hot Chips, August 2021.

[36] 高架扇出桥先进封装架构支持的 AMD Instinct™ MI250X 加速器

R. Swaminathan, M. J. Schulte, B. Wilkerson, G. H. Loh, A. Smith, N. James, “AMD Instinct™ MI250X Accelerator enabled by Elevated Fanout Bridge Advanced Packaging Architecture,” in the International Symposium on VLSI Technology and Circuits, June 2023.

[37] LLNL 与 HPE 将携手 AMD 开发 El Capitan，预计将成为世界最快的超级计算机

J. Thomas, “LLNL and HPE to partner with AMD on El Capitan, projected as world’s fastest supercomputer,” Lawrence Livermore National Laboratory, March 2020, <https://www.llnl.gov/archive/news/llnl-hpe-partner-amd-el-capitan-projected-worlds-fastest-supercomputer>

[38] Triton：面向分块神经网络计算的中间语言与编译器

P. Tillet, H. T. Kung, D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2019

[39] LLaMA：开放且高效的基础语言模型

H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, “LLaMA: Open and Efficient Foundation Language Models”, arXiv:2302.13971 [cs.CL], <https://doi.org/10.48550/arXiv.2302.13971>

[40] 高带宽内存时代

K. Tran, “The Era of High Bandwidth Memory,” Tutorial in HotChips 28, August 2016.

[41] AMD 下一代 “Zen 4” 核心与第四代 AMD EPYC™ 9004 服务器 CPU

K. Troester, R. Bhargava, “AMD Next Generation ‘Zen 4’ Core and 4th Gen AMD EPYC™ 9004 Server CPU,” in HotChips 2023.

[42] 面向百亿亿次级计算的 APU 设计与分析

T. Vijayaraghavan, Y. Eckert, G. Loh, M. Schulte, M. Ignatowski, I. Paul, B. Beckmann, S. Reinhardt, W. Brantley, J. Greathouse, O. Kayiran, M. Poremba, W. Huang, A. Karunanithi, G. Sadowski, V. Sridharan, S. Raasch, M. Meswani, “Design and Analysis of an APU for Exascale Computing,” in the International Symposium on High-Performance Computer Architecture (HPCA), February 2017.

[43] 撞上内存墙：显而易见的事实所带来的影响

W. A. Wulf, S. A. McKee, “Hitting the Memory Wall: Implications of the Obvious,” ACM SIGARCH Computer Architecture News, Volume 23, Issue 1, pp. 20–24, March 1995.

[44] 3D V-Cache：面向 7 nm x86-64 CPU 的混合键合 64 MB 堆叠缓存实现

J. J. Wu, R. Agarwal, M. Ciraula, C. Dietz, B. Johnson, D. Johnson, R. Schreiber, R. Swaminathan, W. Walker, S. Naffziger, “3D V-Cache: the Implementation of a Hybrid-Bonded 64MB Stacked Cache for a 7nm x86-64 CPU,” in the International Solid-State Circuits Conference, February 2022.

版权与商标声明

本文是 ISCA 2024 产业分会论文。

允许为个人或课堂用途免费制作本文全部或部分内容的数字或纸质副本，条件是这些副本不以营利或商业利益为目的制作或分发，并且在第一页保留本声明和完整引文。本文中由作者以外的其他方拥有版权的内容，必须尊重其版权。允许在注明来源的前提下进行摘要。其他复制、再版、发布到服务器或向邮件列表再分发的行为，须事先获得明确许可和 / 或支付费用。许可申请请联系 Permissions@acm.org。© 2024，版权由所有者 / 作者持有。出版版权许可给 ACM。

这是本作品的作者版本，仅供个人使用，不得再分发。正式出版版本发表于 ISCA' 24。（译注：本地原件首页的这一出版说明至此结束。）

AMD、AMD 箭头标志、CDNA、EPYC、Instinct、Infinity Fabric、ROCm、Ryzen 及其组合，是 Advanced Micro Devices, Inc. 的商标。OpenMP 名称和 OpenMP 标志是 OpenMP Architecture Review Board 的注册商标。PyTorch、PyTorch 标志及任何相关标志是 Linux Foundation 的商标。TensorFlow、TensorFlow 标志及任何相关标志是 Google Inc. 的商标。Ubuntu 是 Canonical Ltd. 的商标。Xeon 是 Intel Corporation 的商标。本文使用的其他产品名称仅用于识别，也可能是其各自公司的商标。